Aya Burpee, Brinda Moudgalya

ID Numbers: 2462847, 2337294

EE/CSE 371

December 5, 2025

Lab 6 Report

Design Procedure

In this lab, we implemented a game of checkers. We used the VGA display to display the board and tokens, and used two 16x7 RAMs to store the locations of each of the 12 red and black tokens. As the program uploads, the board is loaded to the VGA display, and the initial values of the token locations, set in two separate .mif files for each the red and black tokens, will be loaded in to create the initial setup on the VGA display as well.

A standard checkers board is a 16x16 array of alternating brown and white tiles, and there exist 12 tokens for each of the two players (red and black). The tokens are arranged on the top and bottom 3 rows of brown tiles. In our implementation, we have the red tokens arranged at the bottom of the board, and the black tokens at the top. The player with the black tokens starts first.

The user can select a token to move by toggling SW[5:3] to set the x-coordinate and SW[2:0] to set the y-coordinate of the checkerboard. If an invalid switch combination is selected (in that there is no token within the selected tile), a signal called invalid_move is pulsed high, which flashes all of the HEX displays on the DE1_SoC. Once selecting a token to move, the user can move it using KEY0, KEY1, KEY2, or KEY3. KEY3 moves the token to the above left tile, KEY2 moves it to the above right tile, KEY1 moves it to the bottom left tile, and KEY0 moves it to the bottom right tile. Based on what token is selected at the time that a KEY is pressed, the system decides whether that move is valid, and either updates the VGA display (and respective RAM blocks) accordingly or does nothing and waits for the user to input a valid combination of SW and KEY inputs.

Our design includes three main components in our top level (DE1_SoC) in addition to the video driver. The first is our ram_update module that implements all of the game logic, which instantiates the submodules ram_update_ctrl and and ram_update_datapath, which are the control and datapath modules for the ASM. The ram_update module performs all of the checks to determine which player's turn it is, if a move is valid, updates our 16x7 RAMs storing token locations, as well as our 64x3 RAMs storing tile encodings (more details later on). The second component is our display_update module. The display_update module takes the x and y outputs from the video driver as well as the tile encoding of the current tile that the video driver is drawing pixels for, and outputs the correct RGB values that need to be drawn at that coordinate.

These RGB values then get fed into the video driver so that it knows what color pixel to draw at the current location. The third component is the 64x3 RAMs that store tile encodings. In each address, which corresponds to each of the 64 tiles on the board, they store one of six possible encodings: 1) empty dark brown tile, 2) empty light brown tile, 3) normal red token, 4) normal black token, 5) KING red token, and 6) KING black token. These values are updated by the ram_update module, and are read by the display_update module.

The ASMD for the ram_update module is shown below in Figure 1.

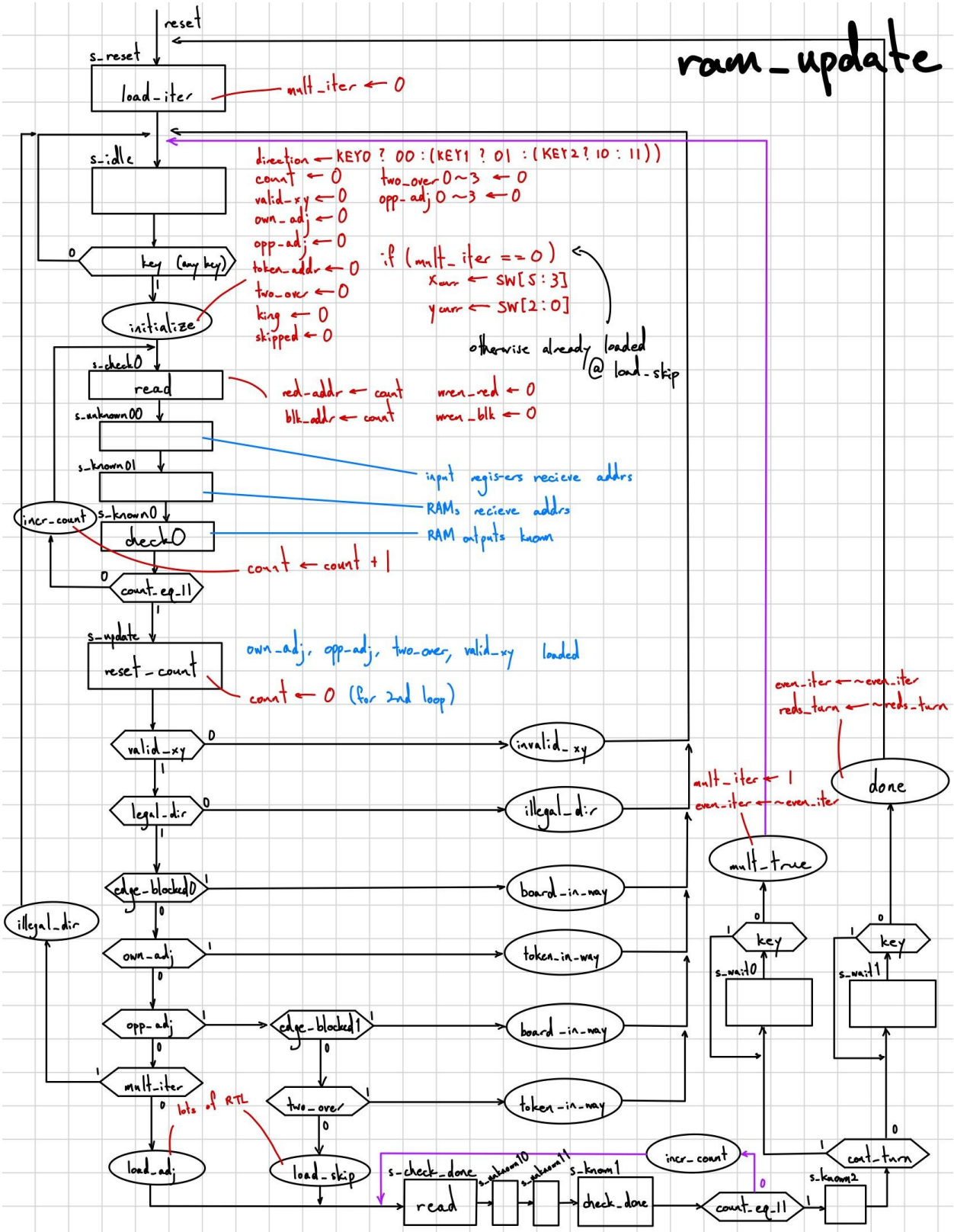


Figure1: ram_update module ASMD

Note on Figure 1: for control signals read, check0, load_adj, load_skip, and check_done, there was too much RTL to fit all of it into the diagram.

In our ASMD, an iteration starts once a player attempts to make a move by pressing a KEY. At that point, our machine enters a loop, scanning through two 16x7 RAMs that store the location of each of the 12 red tokens and the black tokens respectively (black_ram16x7 contains the locations of the black tokens and red_ram16x7 contains the locations of the red tokens). Since there are only 12 red and black tokens each, the last four addresses of the RAMs are left "empty"; the values are set to zero. The word size of 7 includes a 1-bit indication of whether a token is a KING token, 3 bits indicating the x-coordinate of the tile chosen, and 3 bits indicating the y-coordinate of the chosen tile. If the KING bit is high, that means that the token traversed all the way across the board to the last row of the opposite side, which allows the token to now move in all four diagonals. If the token is not a KING token, then it can only move either up or down (black tokens can only move downwards and red can only move upwards). Once the RAMs are scanned, the relevant status signals are produced to determine whether the player's move is valid or not. These status signals include valid_xy, legal_dir, edge_blocked0, own_adj, opp_adj, edge_blocked1, and two_over as seen in Figure 1.

The valid_xy signal is true if the location selected by the switches contains a token of the current player. The legal_dir signal is true if the direction selected by which KEY was pressed is a direction that the selected token is allowed to move in. For example, if an upward direction was selected to move a normal black token (which can only move down) or a downward direction as selected to move a mornal red token (which can only move up), legal_dir would evaluate to FALSE. The edge_blocked0 signal checks whether there is an edge of the board directly adjacent to the token in the direction that the player selected to move in. The own_adj signal checks whether there is a token belonging to the current player that is adjacent to the selected token in the direction that the player selected to move in. The opp_adj signal checks whether there is an opponent's token adjacent to the selected token in the direction that the player wants to move in. edge_blocked1 checks whether there is a board edge two spots over in the direction that the player selected to move in. The two_over signal checks whether there is a token two spots over in the direction that the player wants to move in. In the case that the current player is preparing to jump over an opponent, this edge_blocked1 and two_over signals would prevent the player from making that move.

All of the above signals are used by the control module to determine whether the player is allowed to make their desired move.

Although we do not shows these as separate signals during the game on the DE1_SoC, for debugging in ModelSim, we created 4 different one-pulse error indication signals in our control module, including invalid_xy, illegal_dir, board_in_way, and token_in_way to indicate the reason for a move being rejected. On the DE1_SoC, all these separate signals have been combined with an OR gate to create the invalid_move signal connected to the HEX displays.

Moving on in the ASMD, once a move is deemed valid, either load_adj or load_skip is asserted as control signals, and the selected token is either moved to the adjacent spot in the selected direction (if the adjacent spot is empty), or moved two spots over in the selected spot, if the player skips over an opponent's token. The player skips over an opponent's token if opp_adj is TRUE and both edge_blocked1 and two_over are FALSE. During this step, new values are written into the RAMs with new player locations. For skipped-over tokens, we set the value in our 16x7 RAMs for that token to be 7'b0, which corresponds to the top left corner of the board (which is an invalid location), to indicate that that piece is out of the game.

The ASM then enters it's second loop which scans the updated 16x7 RAMs. This loop has 2 main functions. The first function is to determine whether the player's turn continues. The player's turn only continues if in the first turn, it skipped over an opponents token, and all of the checks pass to be able to skip over another opponent token. The second function is to update the tile encodings in the 64x3 RAMs so that the tiles are erased and redrawn accordingly on the actual VGA. Then the ASM iteration ends. If the current player's turn continues, the logic for the next iteration through will be slightly different as seen in the Figure 1. Otherwise, the signal keeping track of who's turn it is toggles, indicating that it is the next player's turn.

The logic to actually load the token positions to the VGA in a way the user can see was done by using the 64x3 RAMs and two other modules: display_update and video_driver. As mentioned above, the general idea was that for every x, y value that the video driver provides (which indicates which pixel on the VGA it is currently writing to), we the display_update module would supply the correct RGB value. To determine which RGB values to supply, the display_update module reads from the 64x3 RAMs at the address 8*(y of board) + (x of board), where y of board and x of board range from 0 through 7, to access the tile encoding for the current VGA location. Since we want the board to be centered on the VGA, for the video driver x-values 0-79 and 560-679, we output RGB values 0 to fill those locations with black. Otherwise, the output RGB values correspond to the tile encoding.

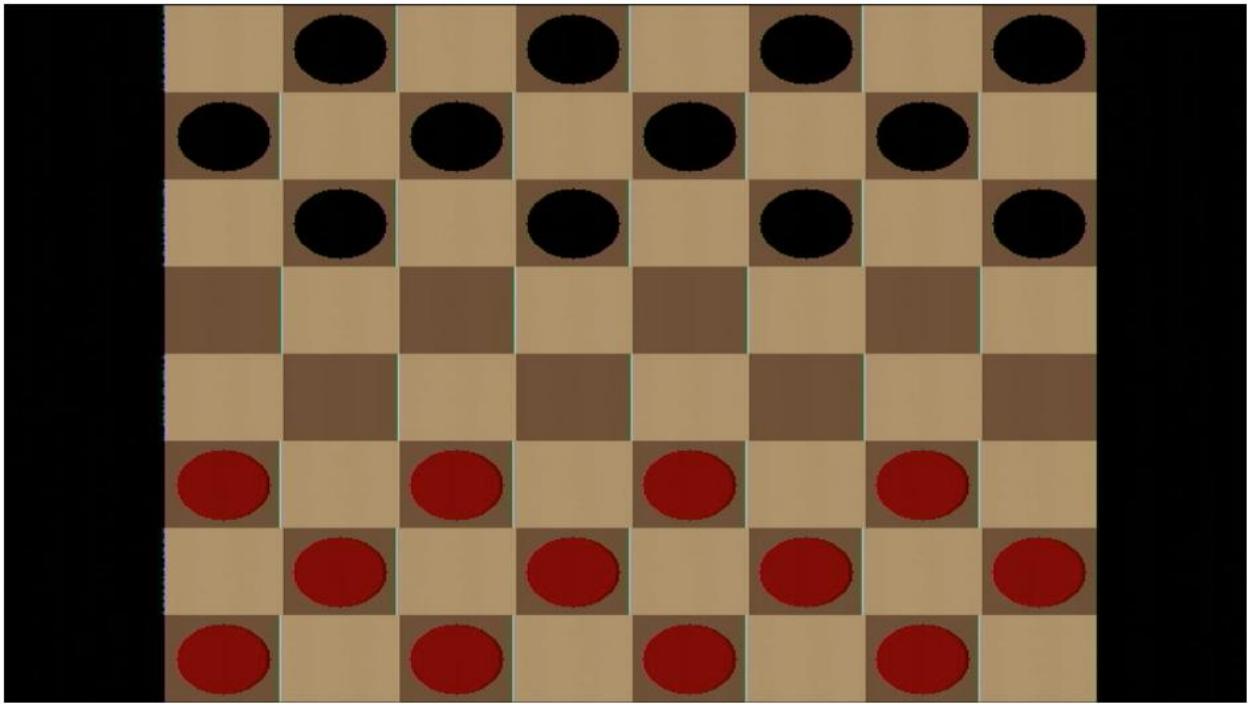
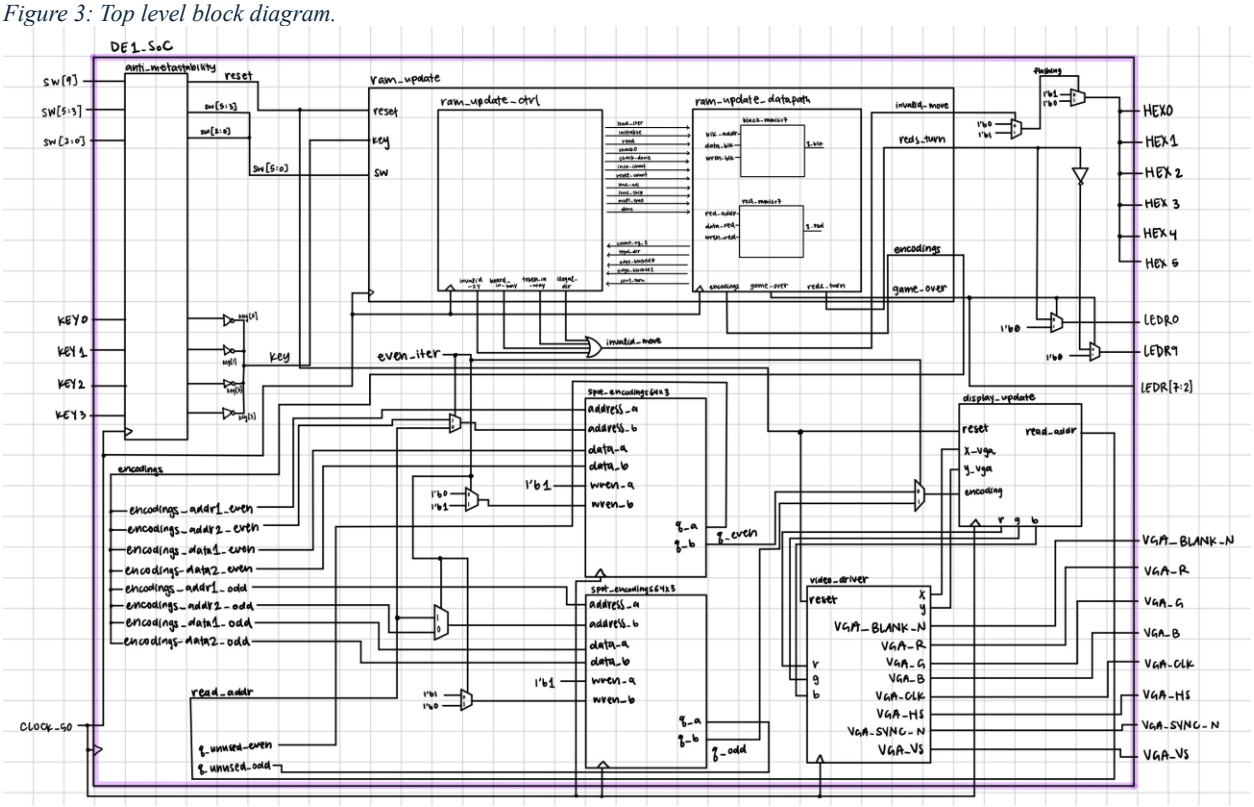


Figure 2: Initial game setup on VGA

At reset, a down counter for each player gets loaded to 12, and each time a token gets captured, the corresponding counter gets decremented.

The game ends when all tokens of one of the players are captured (one of the counters has a value of 0). The top-level block diagram is given below:

Figure 3: Top level block diagram.



Results

The logic from the ASMD chart was then implemented in SystemVerilog code, and the testbenching of different modules are included below along with an explanation of the signals visible. The game logic follows almost exactly from the idea presented in the Lab 6 Proposal, with minor updates for functionality reasons. For example, the proposal outlined using SW[2:0] for x-coordinates and SW[5:3] for y-coordinates, but in the final lab, we ended up switching the two, so SW[5:3] represented the x-axis and SW[2:0] represented the y-axis. Additionally, in the proposal, we modularized every check for making sure a move is valid, but as we worked on the coding of the lab, we ended up being able to clearly put all the game logic within just a single ram_update module, which included both a control and a datapath. We did include a separate module for implementing the VGA display. The other major change was adding two 2-port RAMs for a 64x3 storage of encodings for each tile. The reason we are using two 64x3 RAMs instead of one is because our update_rams module writes to both ports at once when updating the values, but the display_update module must read from a different address to access the correct encoding, and we wanted to prevent a read-write overlap on the same clock cycle. Therefore during even iterations of our ASM, we write to the first 64x3 RAM and read from the second, and on odd iterations we write to the second RAM and read from the first. All other significant

logic is consistent with the proposal, such as instantiating two 16x7 RAMs to store red and black token locations and using a number-of-tokens-remaining system to decide game_over logic.

ram_update.sv

This module instantiates the ram_update_datapath and ram_update_ctrl blocks. Since the control and datapath work together to update control signals and datapath signals respectively, we testbench both together by testbenching the overall ram_update module. Figures 4 and 5 depict the waveforms used for ensuring functionality of the RAM modules that are used to store the locations of the tokens.

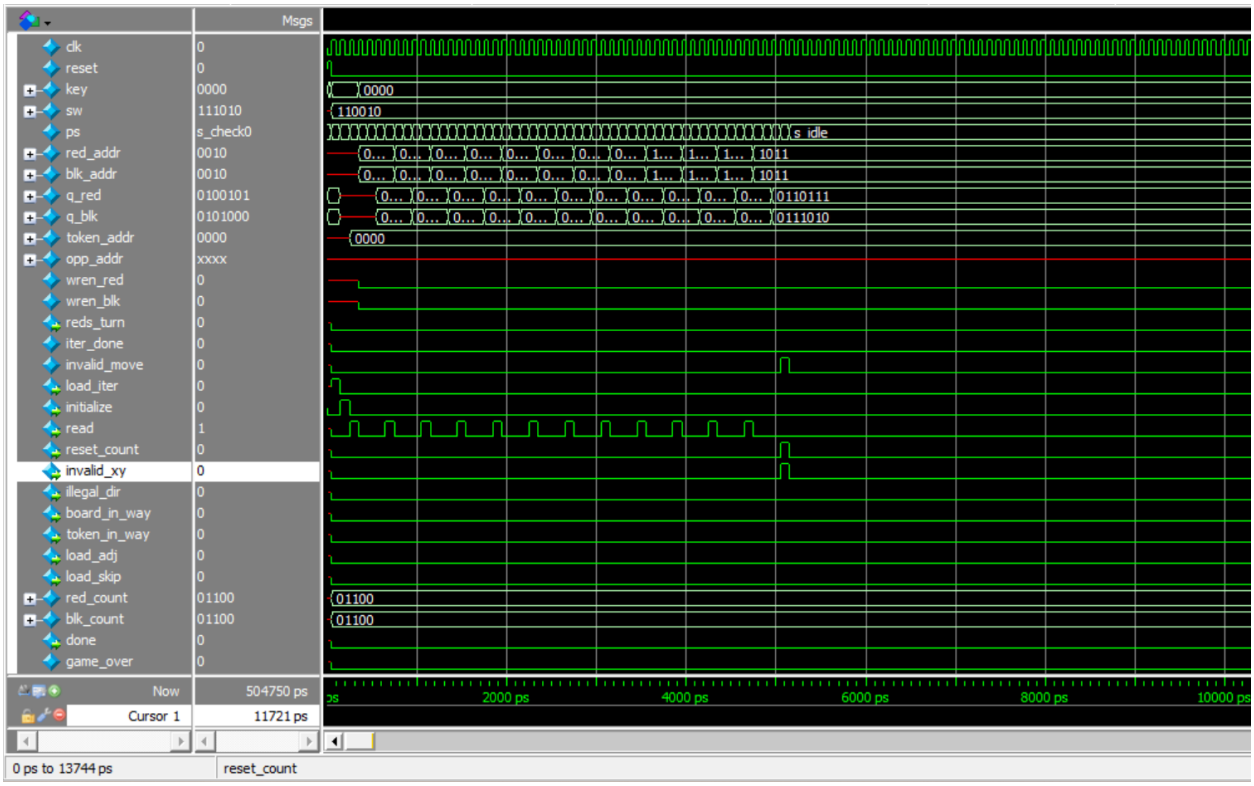


Figure 4: Selecting a tile without a token in it will raise an invalid_xy error.

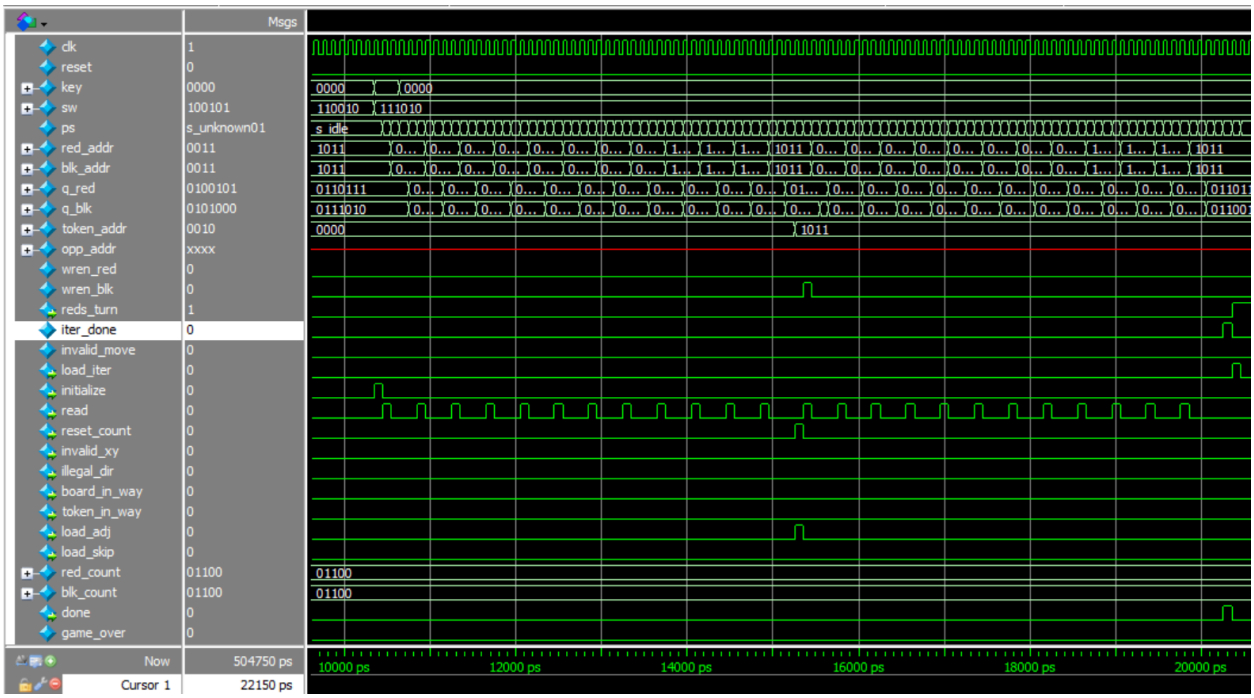


Figure 5: Selecting a tile with a valid token will move the tile in the direction specified by the KEY – in this case, down left.

As seen in the above Figures 4 and 5, when the user selects a tile without a token present within it, the invalid_xy signal pulses, and no token is moved. However, when the user selects a valid switch combination, upon KEY press, if the direction specified by the KEY is a valid direction, then the token moves to that spot on the board (and its location is updated in the corresponding RAM).

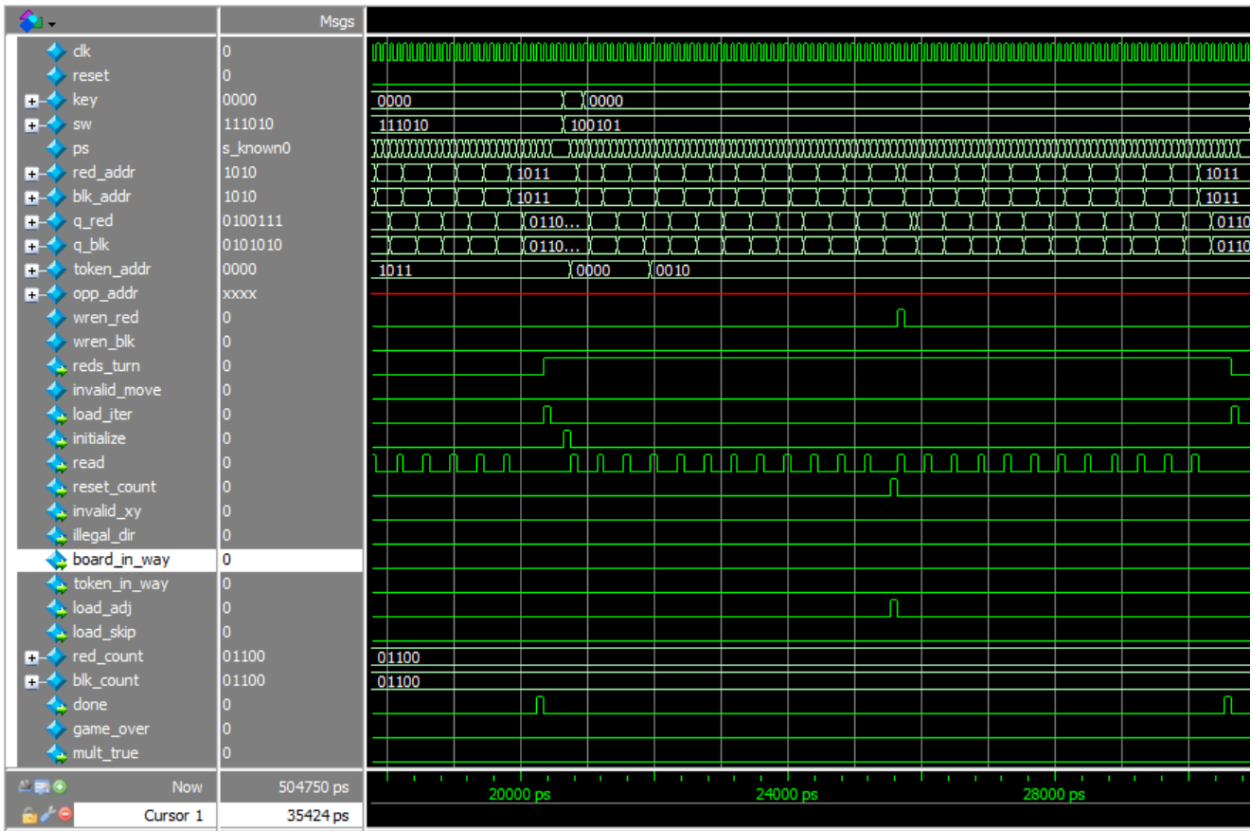


Figure 6: Valid token move, up left.

After a valid token move, the board switches over to the opposite player. This is seen in Figure 6 by making a valid move while reds_turn is high, and confirming that at the end of the move, reds_turn automatically becomes low so the opposite side can start playing.

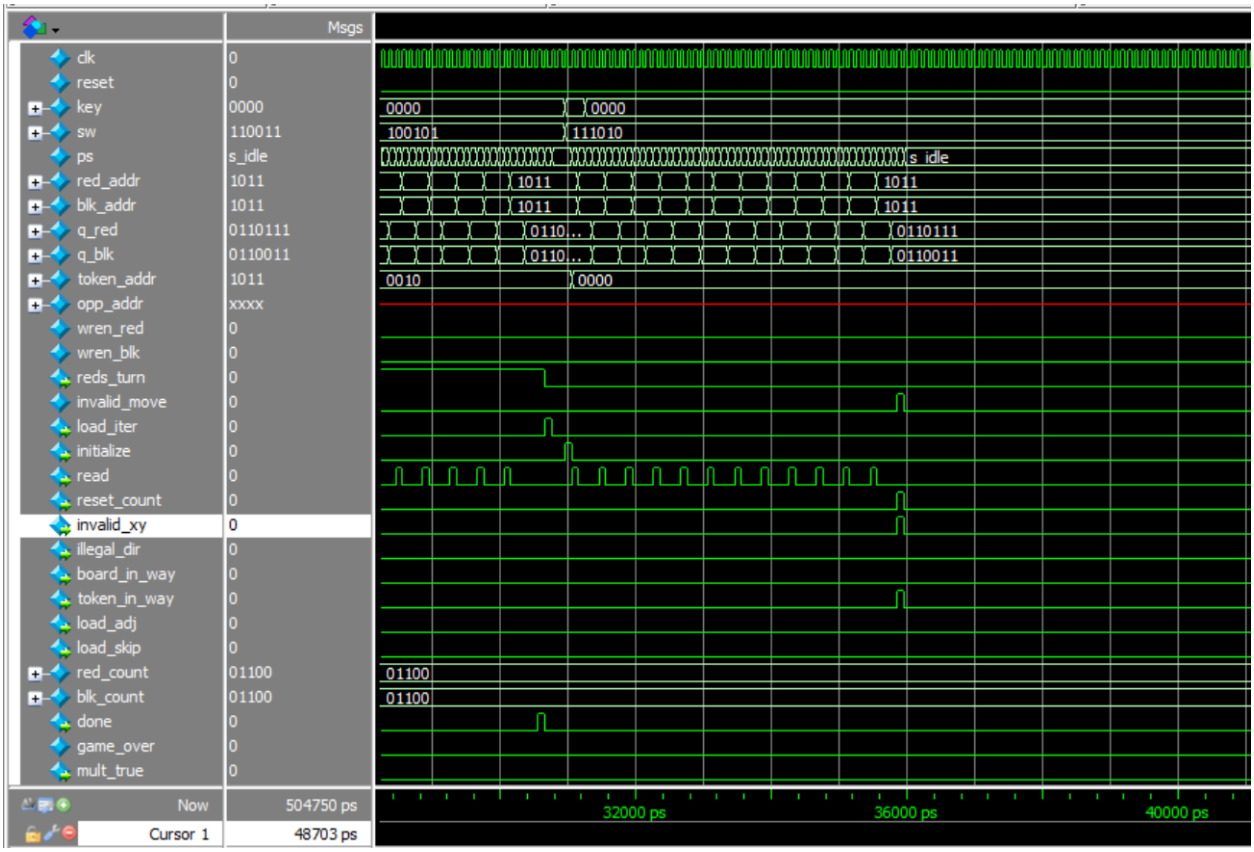


Figure 7: Invalid_xy signal is pulsed high because we attempt to move a token from a spot that is empty.

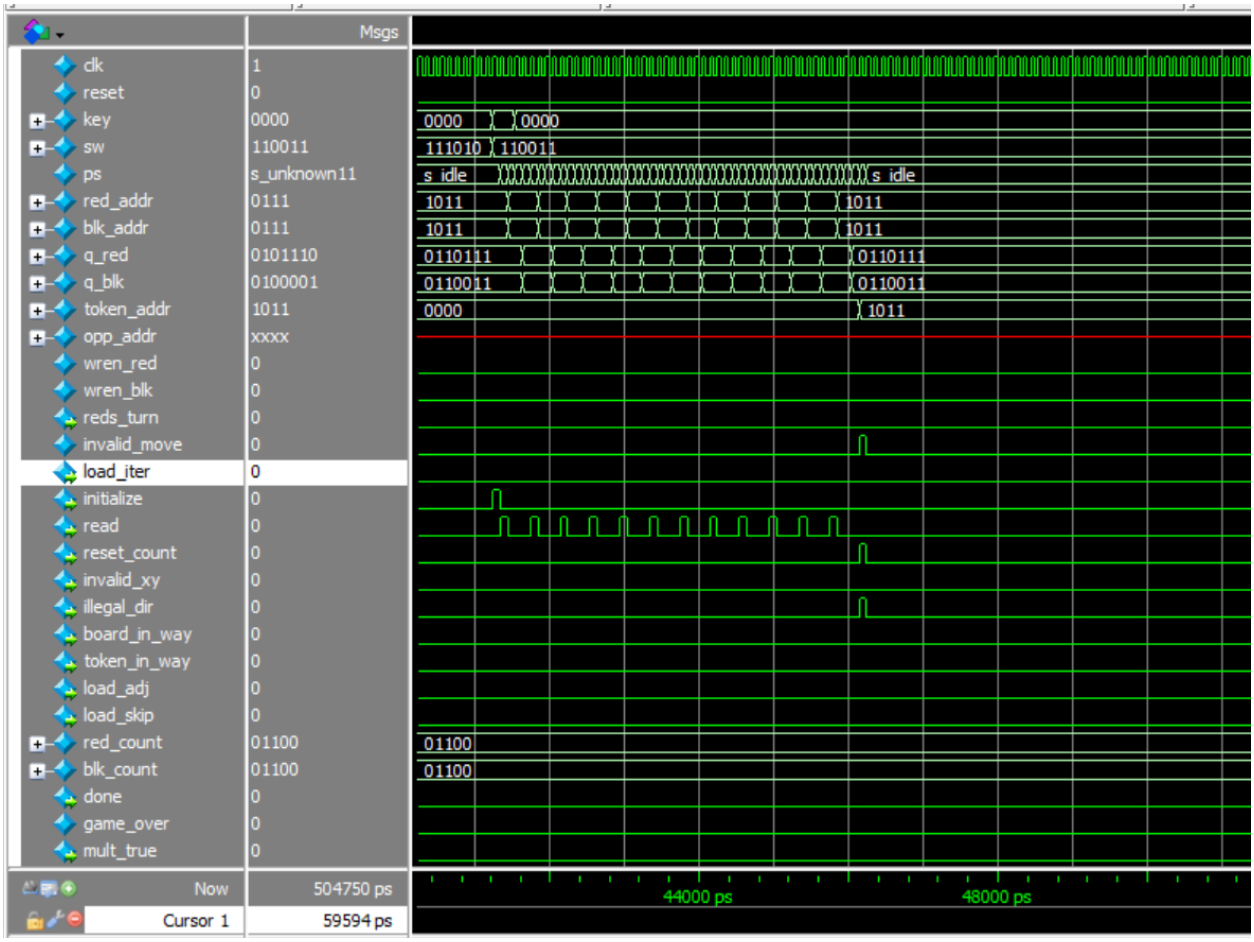


Figure 8: Illegal_dir signal is pulsed high because we attempt to move a token in a direction that is not allowed (moving a black non-king token upwards).

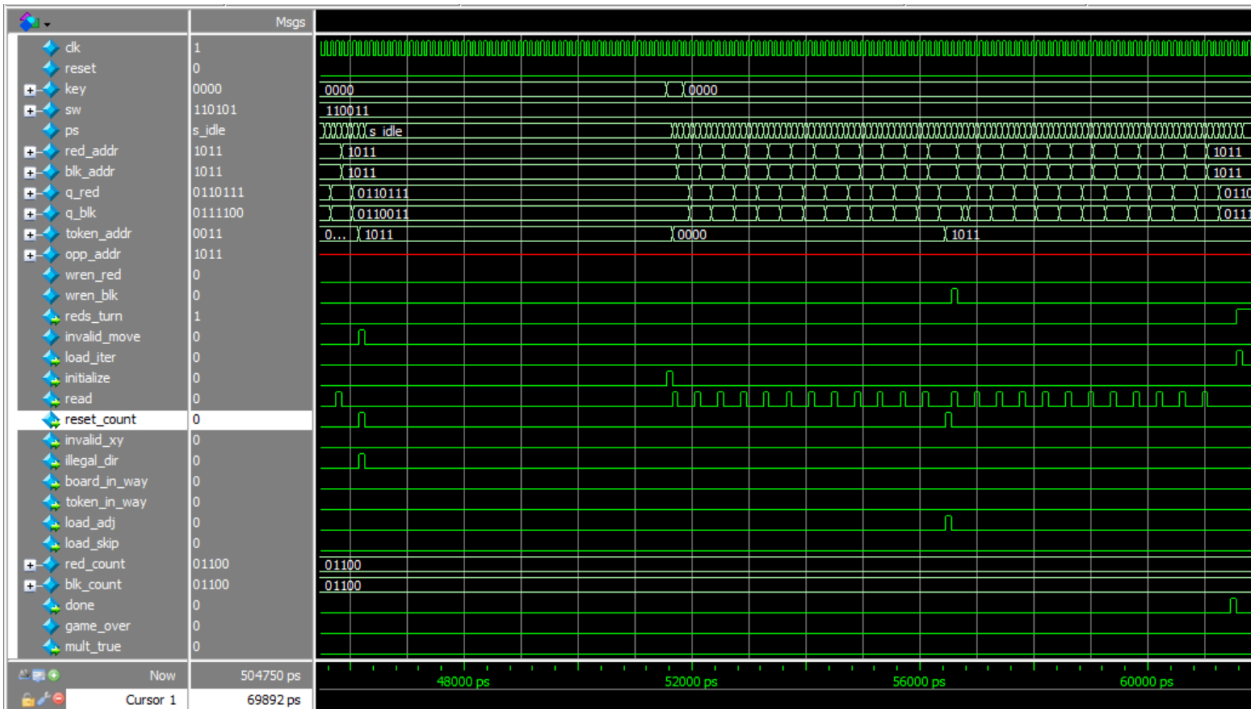


Figure 9: Valid move, down right.

We tested the system to make sure that, once a token is moved from the starting spot, the location in the RAM also changes such that trying to move a [hypothetical] “token” from a spot where a token *used to* exist will raise the invalid_xy signal and indicate error. This is shown in Figure 7. We also tested to make sure that a token cannot move backwards (if it is not a KING token), as shown in Figure 8. Since we are currently working with the black tokens during this turn, attempting to press KEY2 to move the token upwards should not work, since black tokens can only move downwards. The waveforms support this. Again, once a valid move is made, the game switches over to the other player, as seen in the toggling of reds_turn at the very end of the waveforms in Figure 9.

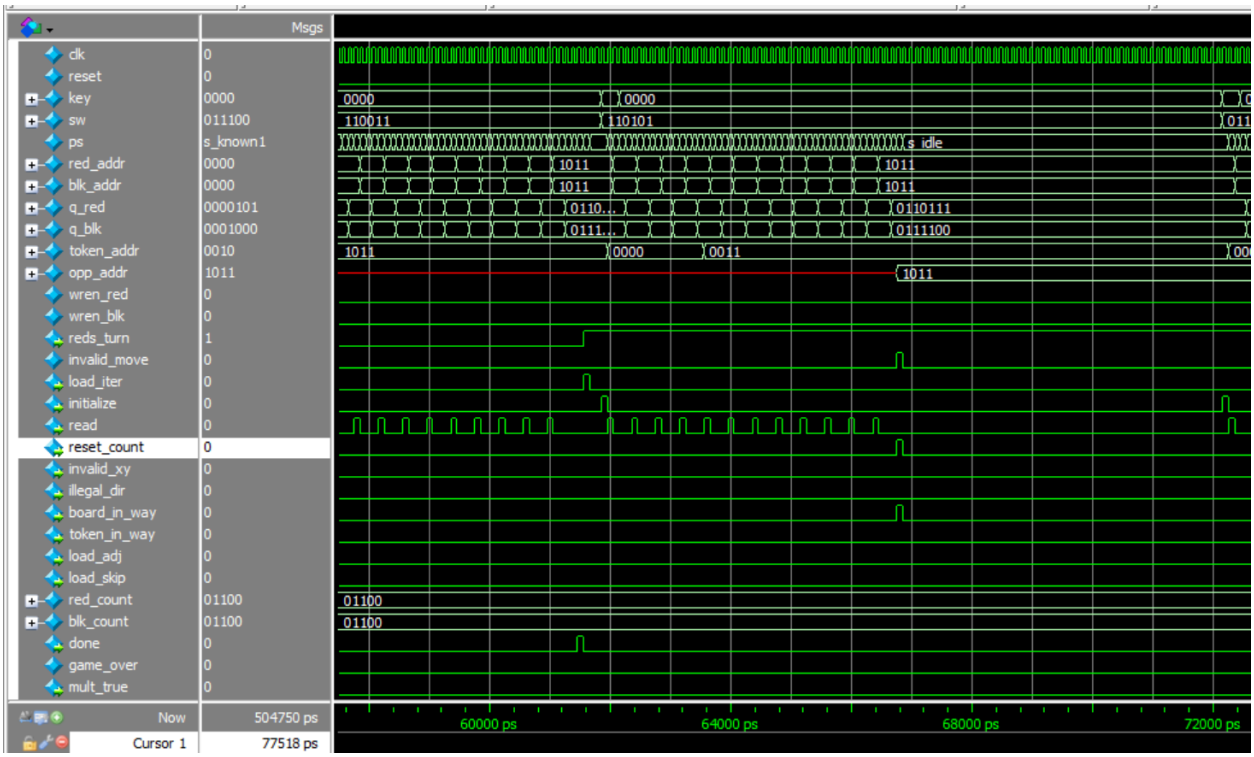


Figure 10: Test whether a token can move if there is an opponent's token blocking that spot.

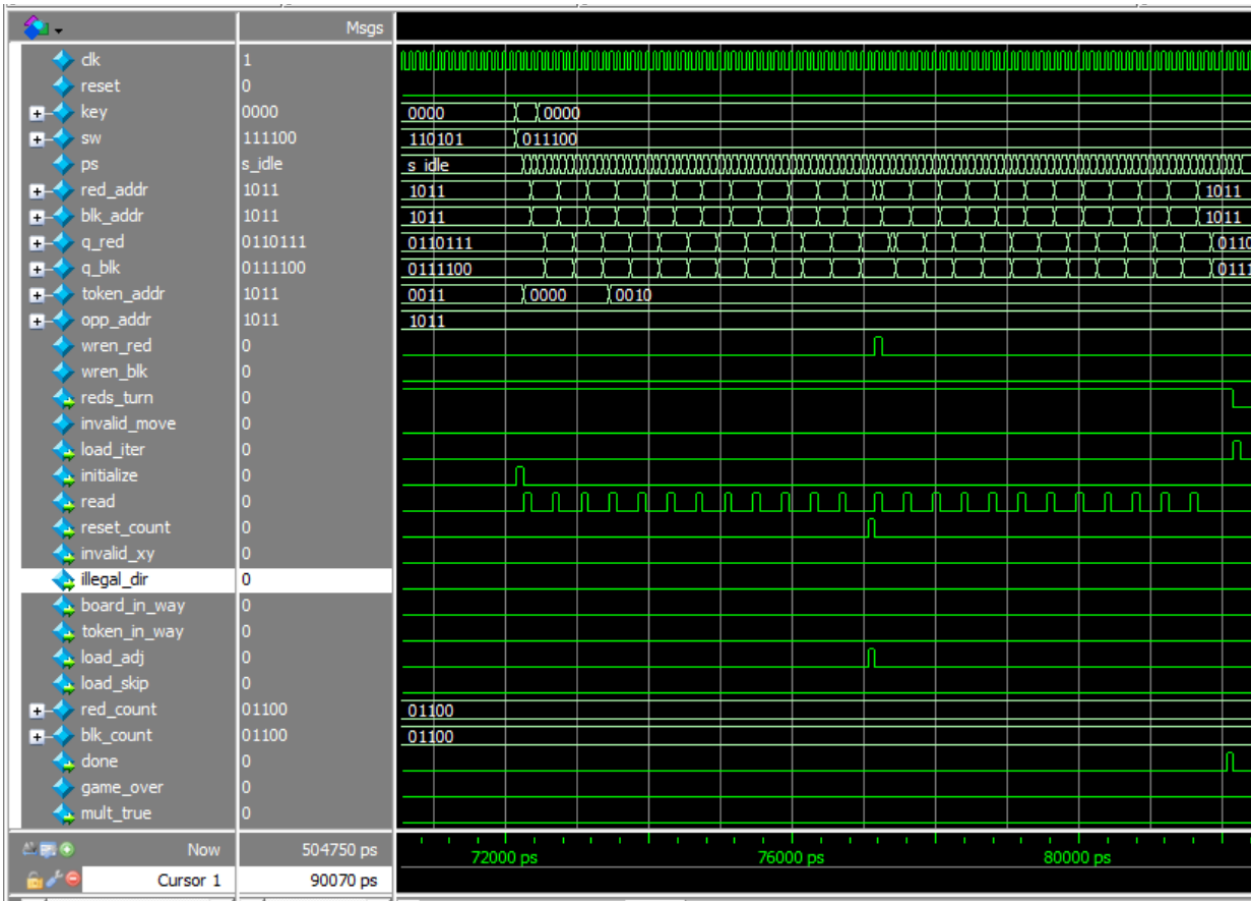


Figure 11: Valid move, up right.

In Figures 10 and 11, we test whether a token can move when the tile in the direction specified already contains a token, thereby blocking the current selected token from moving to that spot. This will raise the board_in_way signal, and the invalid_move signal is pulsed high. We make a valid move to an empty spot, and control switches over to black.

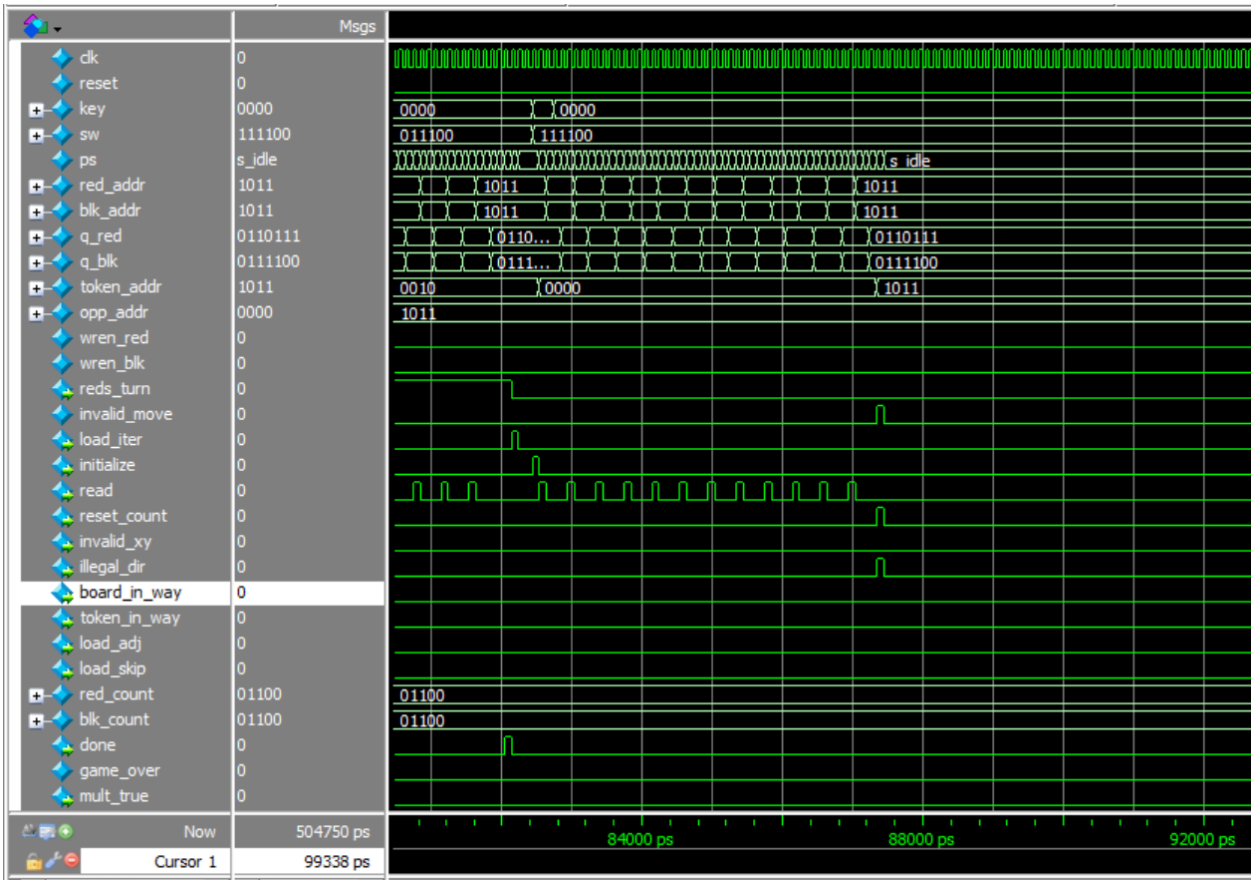


Figure 12: Try to move black token upwards – invalid because it's not a KING token.

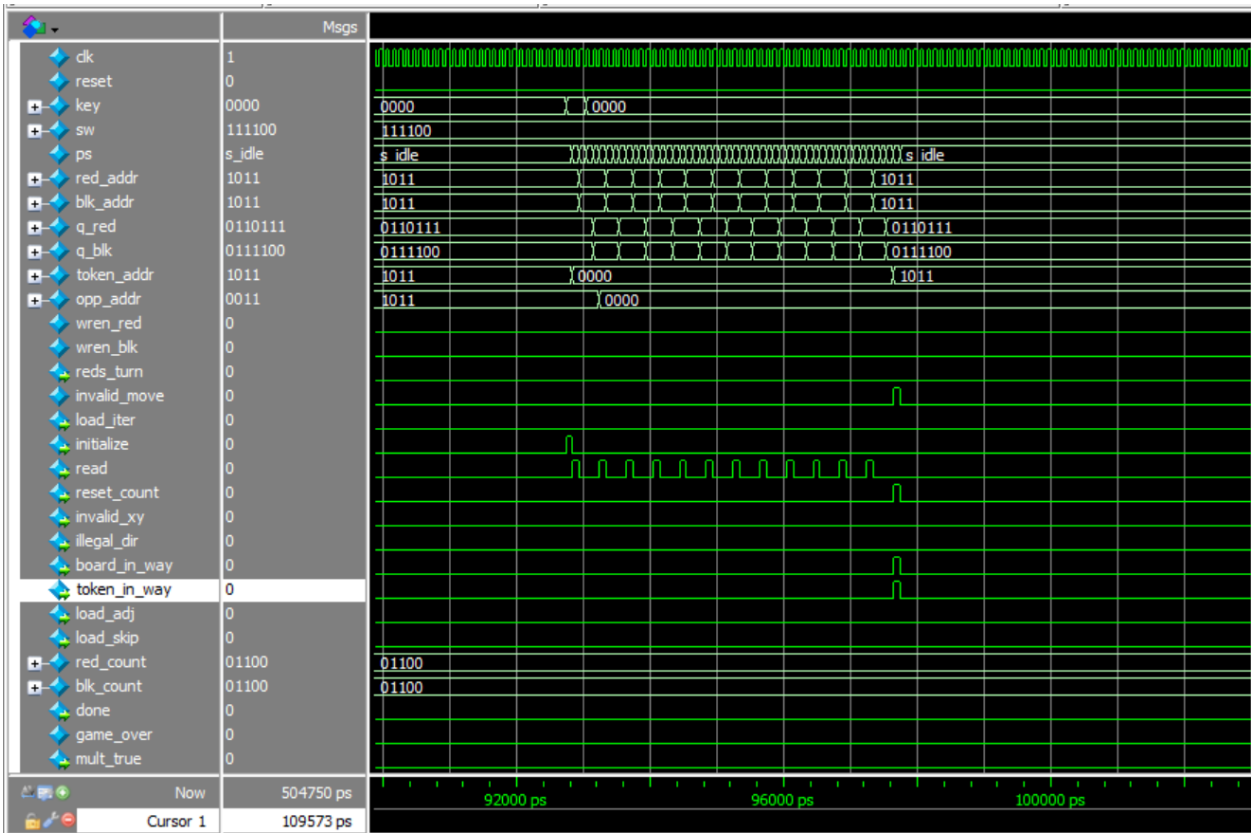


Figure 13: Invalid move because the edge of the board is in the way.

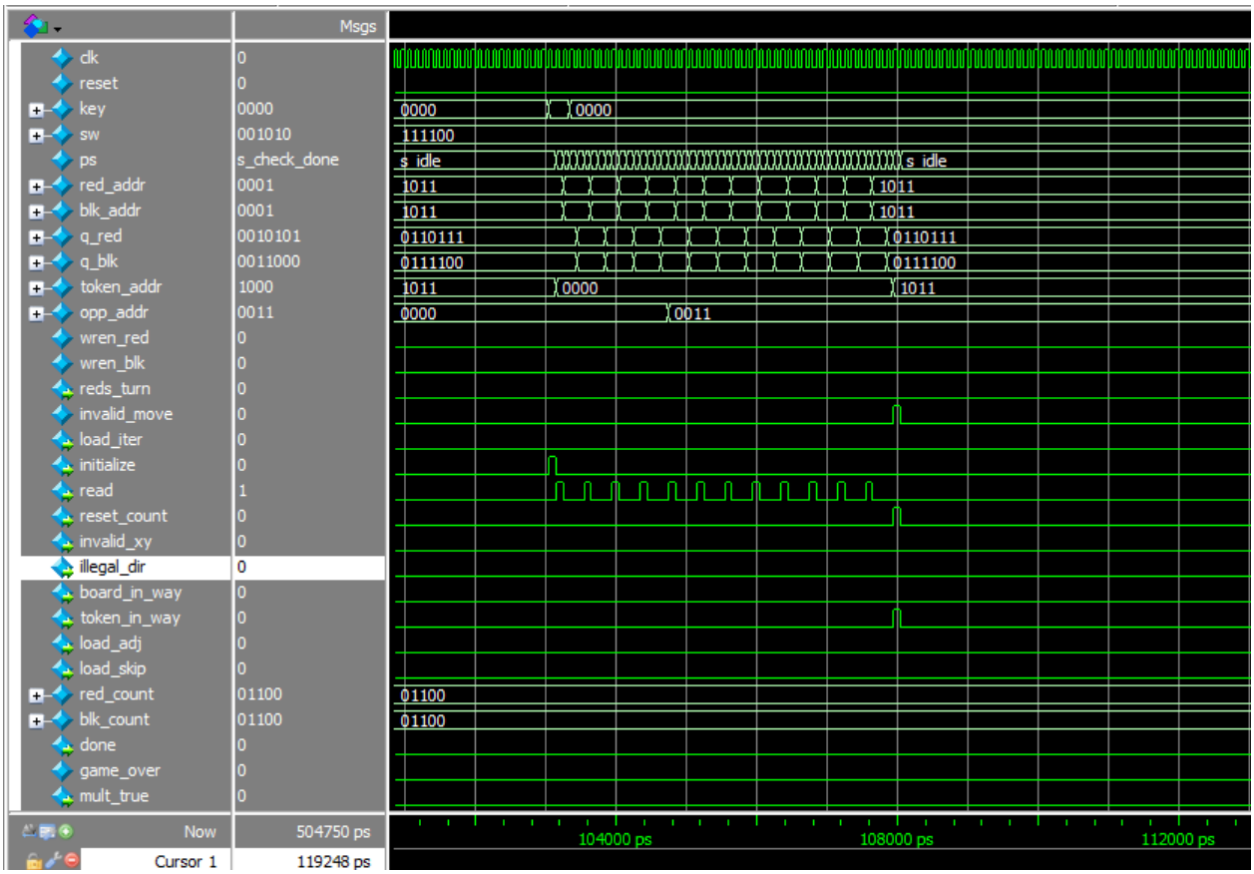


Figure 14: Invalid move – blocked by a token.

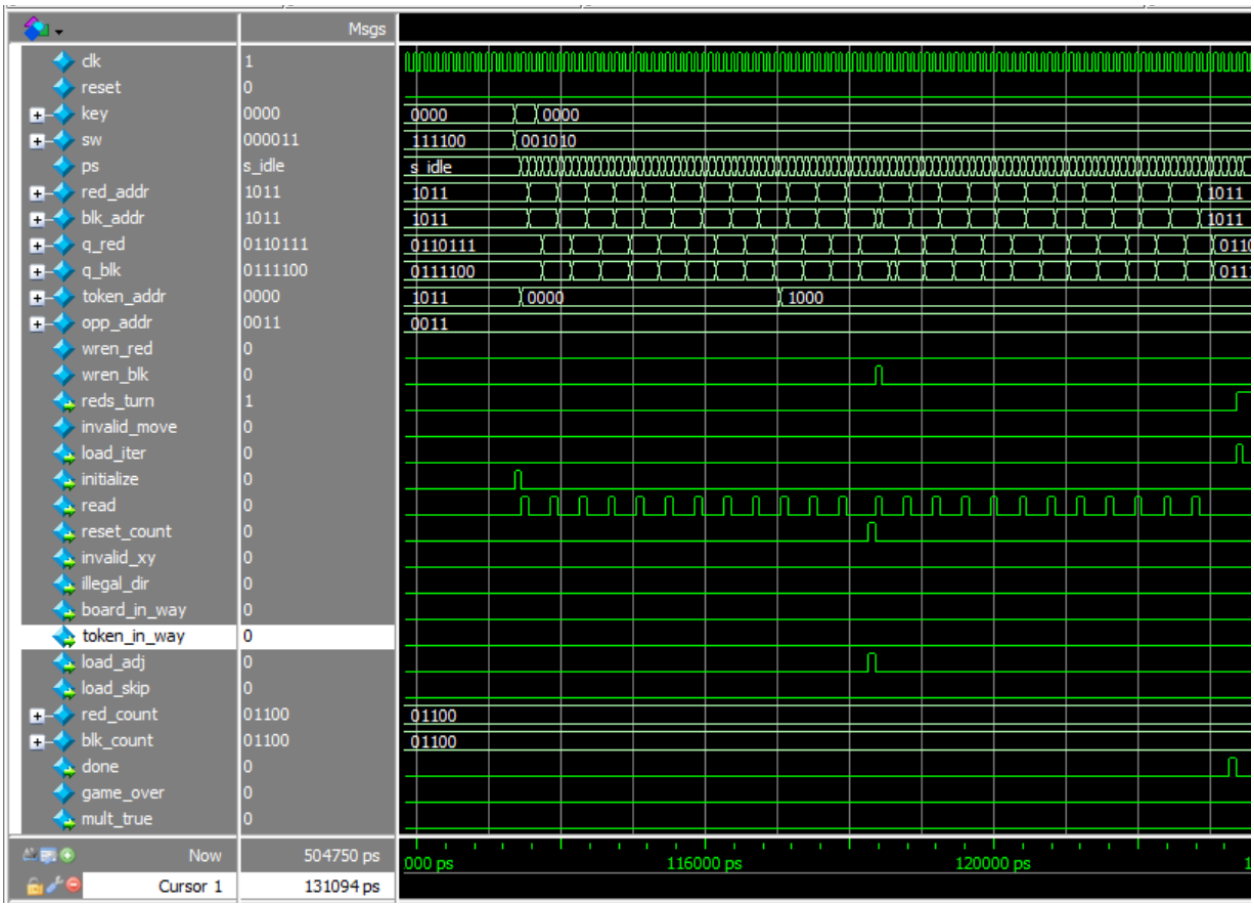


Figure 15: Valid move.

The series of waveforms shown in Figures 12 through 14 show more error testing. In Figure 12, we try to move the black token in an upwards direction by pressing KEY3. Because it is not a KING token, the token does not move, and illegal_dir is pulsed high. In Figure 13, we then try and move it down and to the right, but since the token is already at an edge tile, it cannot move and board_in_way is pulsed high. Then, in Figure 14, we try and move it down left, which is blocked by a red token, so token_in_way is pulsed high. We then will make a valid move (Figure 15) and switch control to red.

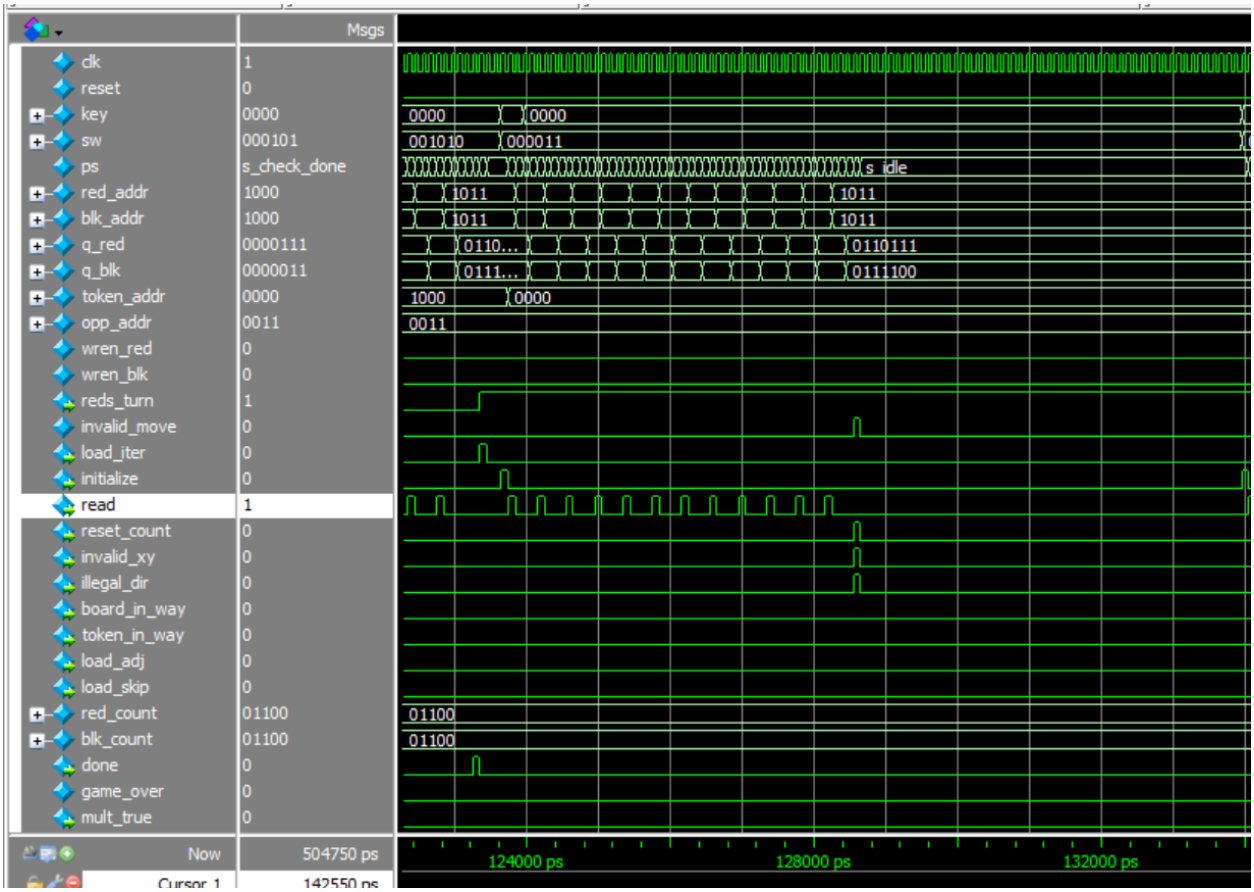


Figure 16: Attempting to move a black token during red's turn.

In the above Figure 16, we see that attempting to move the wrong color token (from a spot that is valid in the black RAM but NOT in the red RAM) will not work, and invalid_xy is pulsed high since we are only looking at tokens that exist in our specific red-token RAM.

After a few rounds of correct playing of the game, we run a few last tests of the RAM module. First, we test the capture logic when it is black's turn.

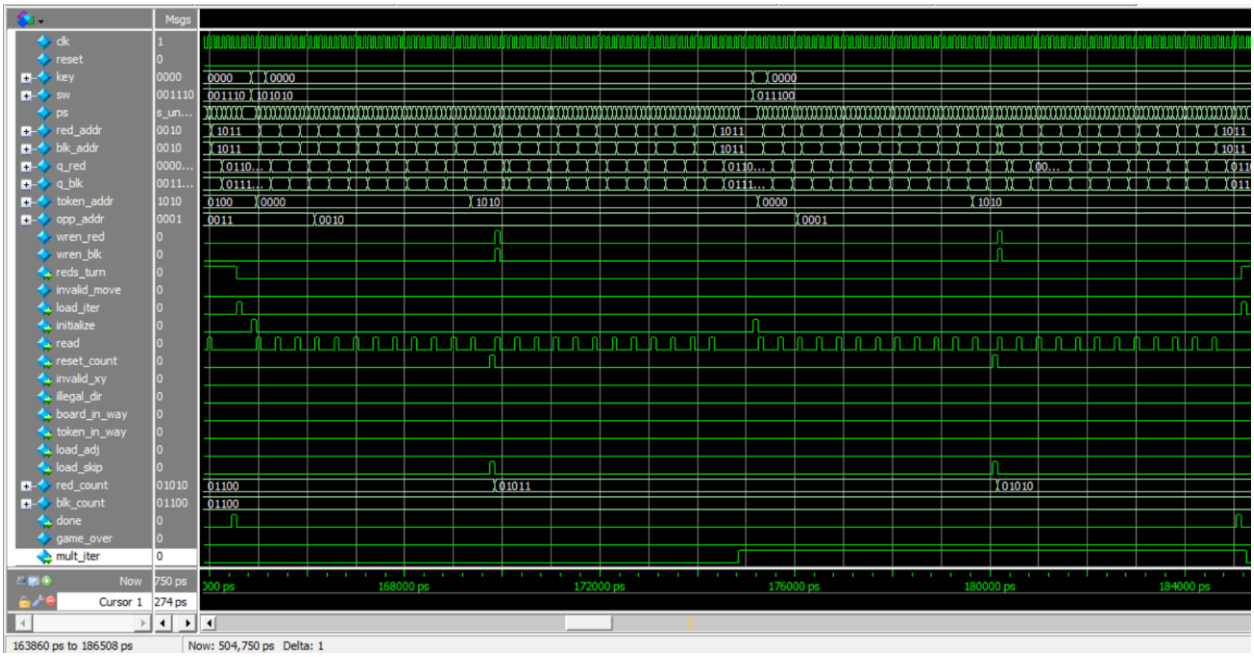


Figure 17: After one capture, mult_iter goes high, allowing the same token to move a second time.

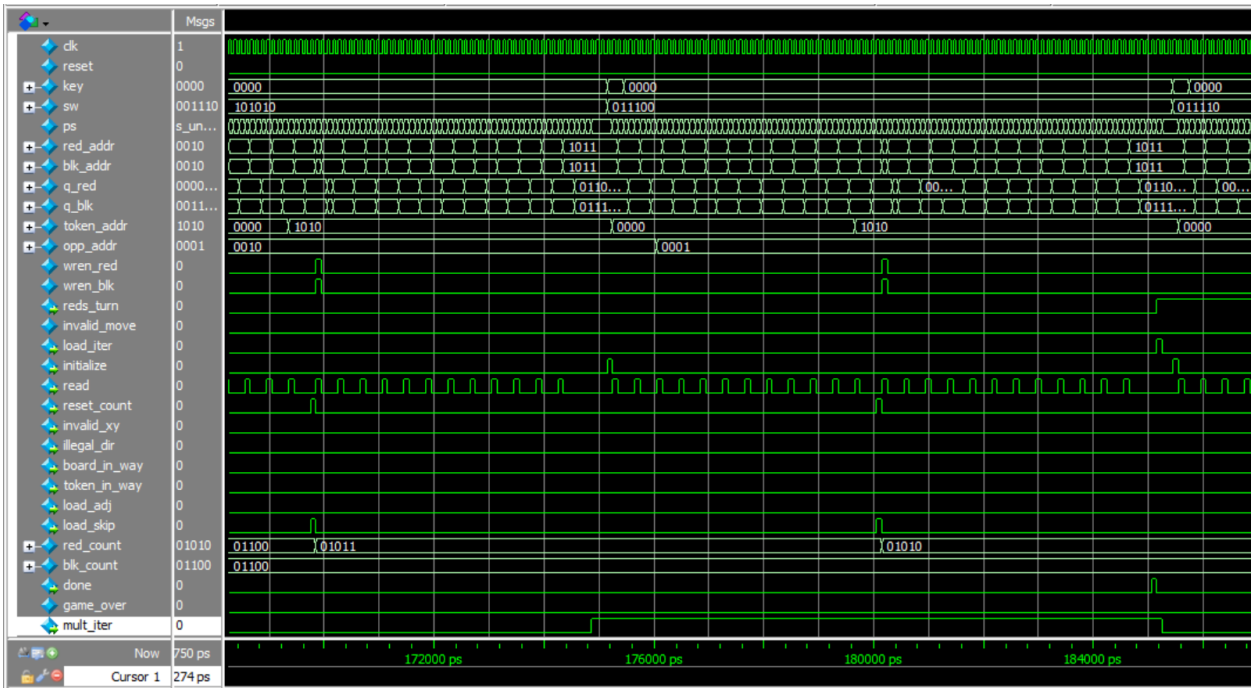


Figure 18: After a second capture, the token cannot make any more moves, and mult_iter goes back down to low.

In Figure 17, it is black's turn, and we select a token that could potentially capture two consecutive tokens. By specifying a direction of KEY1 (bottom left), the selected token sees a red token, and then an empty spot. This allows the selected token to capture the red token. As we can see from the waveforms, both wren_red and wren_black go high. This is because we need to "delete" the captured token (by setting its location in RAM to 7'b0000000, which is an invalid

location for tokens since it is a white tile, and therefore wouldn't mess with game logic), as well as change the black token's location in RAM as usual. After the iteration, mult_iter goes high. This allows the same black token to be moved again in the direction specified by the KEY press.

Then, in Figure 18, the same thing happens, where there is an empty tile directly adjacent to the adjacent red token, which the same black token can capture. After these two captures, there is nowhere left for the selected token to go, so black's turn must end. This is confirmed by mult_iter now being driven low.

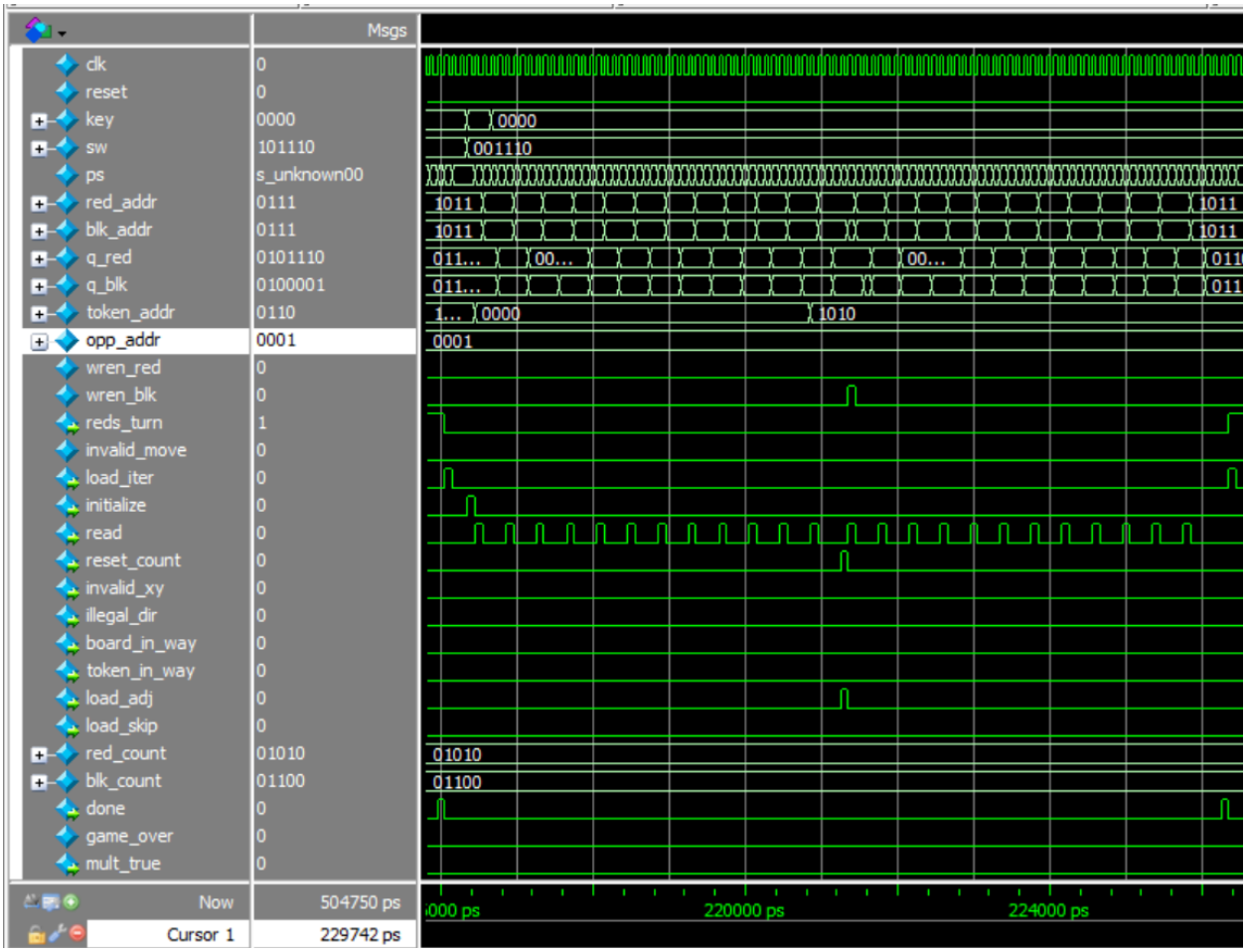
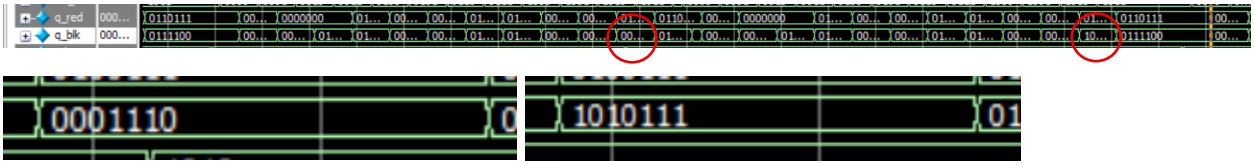


Figure 19: Selected token becomes a KING token.

After a few more valid moves from both players, it is black's turn again. It is currently located in the second-to-last row of the board, and there is an empty tile to its bottom right. Pressing KEY0 allows the token to move to that empty slot, and since the black token has now traversed the entire board to the last row of the opposite side, it becomes a KING token. This move is seen in Figure 19. The MSB in the black RAM corresponding to the token becomes a 1'b1 (as seen in the zoomed-in screenshots below, which show the RAM locations of each of the 12 tokens in RAM).



To clarify, we are looking at the MSB of these two, which are the 11th of the 12 tokens. Once the token moves from x = 001, y = 110 to x = 010, y = 111, because the token has traversed all the way across the board, the KING bit changes from a 1'b0 to a 1'b1.

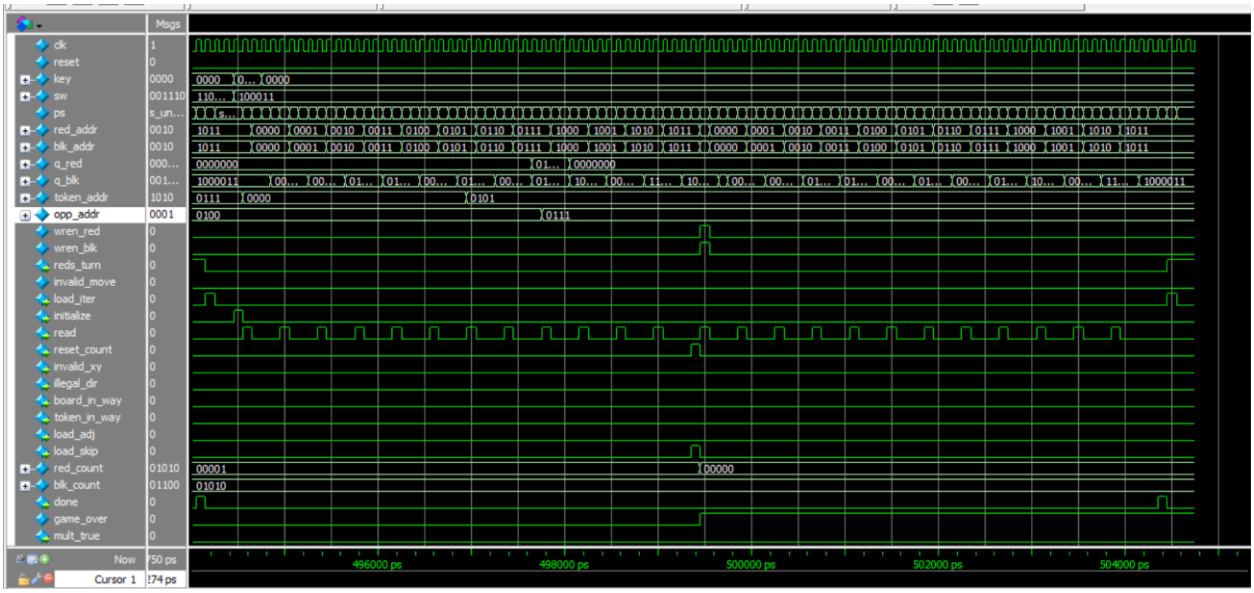


Figure 20: Game_over logic.

The final test was testing the game_over logic. This is seen in Figure 20. The game_over signal is set to 1'b0 unless red_count or blk_count (the number of tokens of each color present on the board) equals 0, in which case it is pulled high.

To clarify, the signals we are looking at are red_count and game_over. The red_count signal goes down from 5'b00001 to 5'b00000, and this immediately toggles the game_over signal to high. The game_over signal is used later to implement game-over logic on the VGA.

Running these tests allowed us to test the functionality of the ram_update modules. Since the ram_update module implements the ram_update_datapath and ram_update_ctrl modules, we tested both by testing the one main module.

Figures 21 to 24 below shows the flow of the steps we took when testing the game. Figure 21 only shows the valid moves, so it shows how we tested the logic for tokens to become KING tokens and the game_over logic.

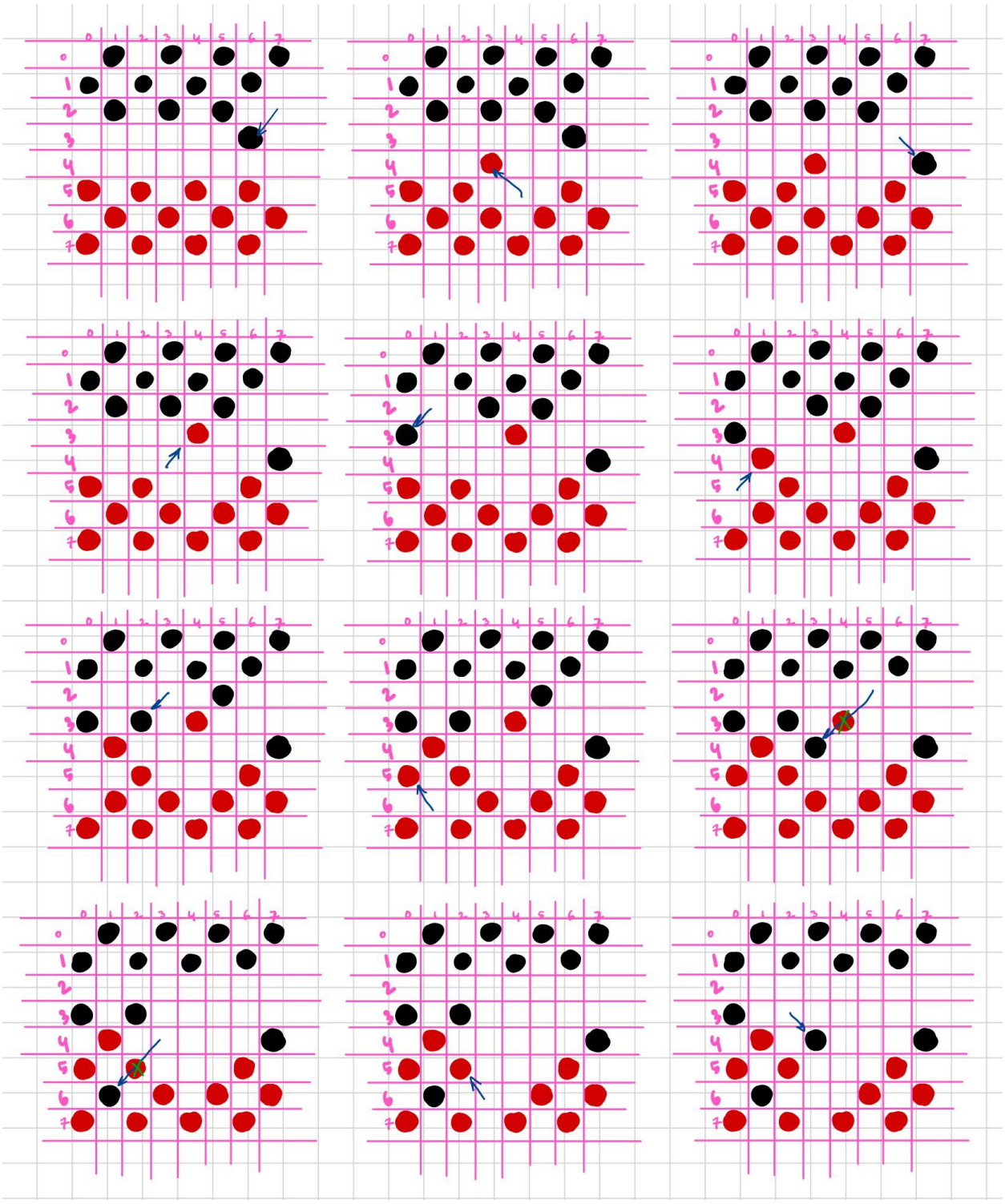


Figure 21

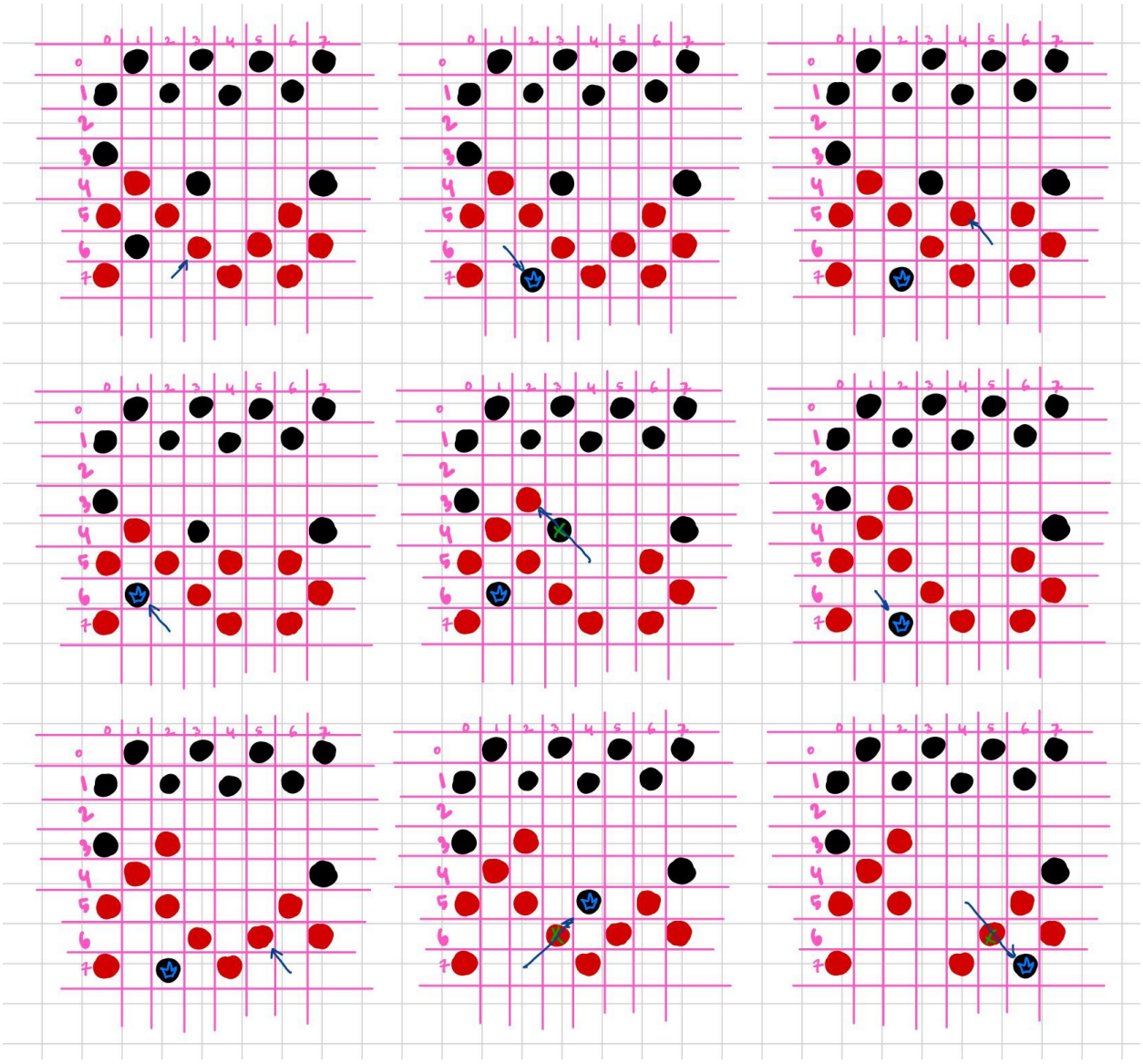


Figure 22

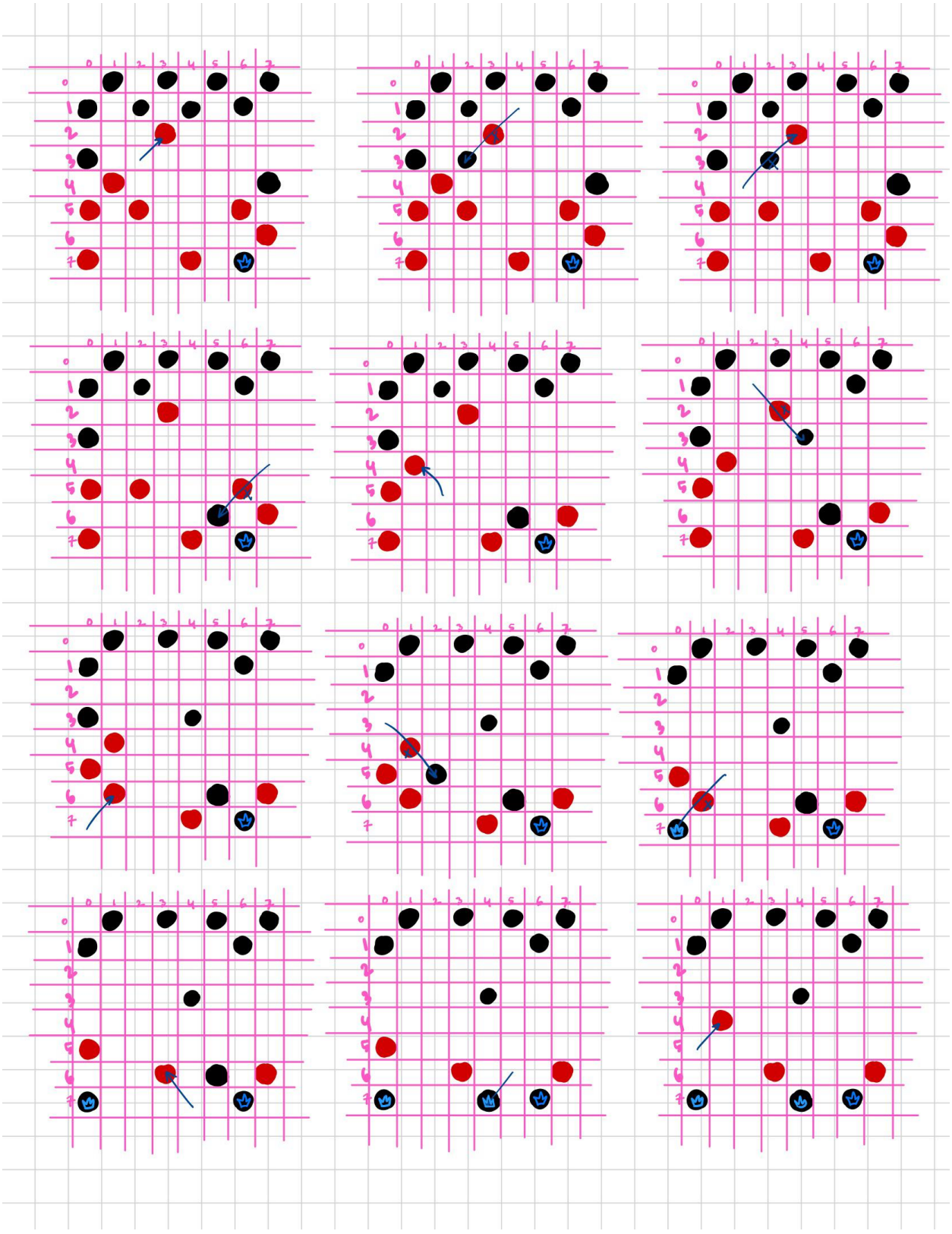


Figure 23

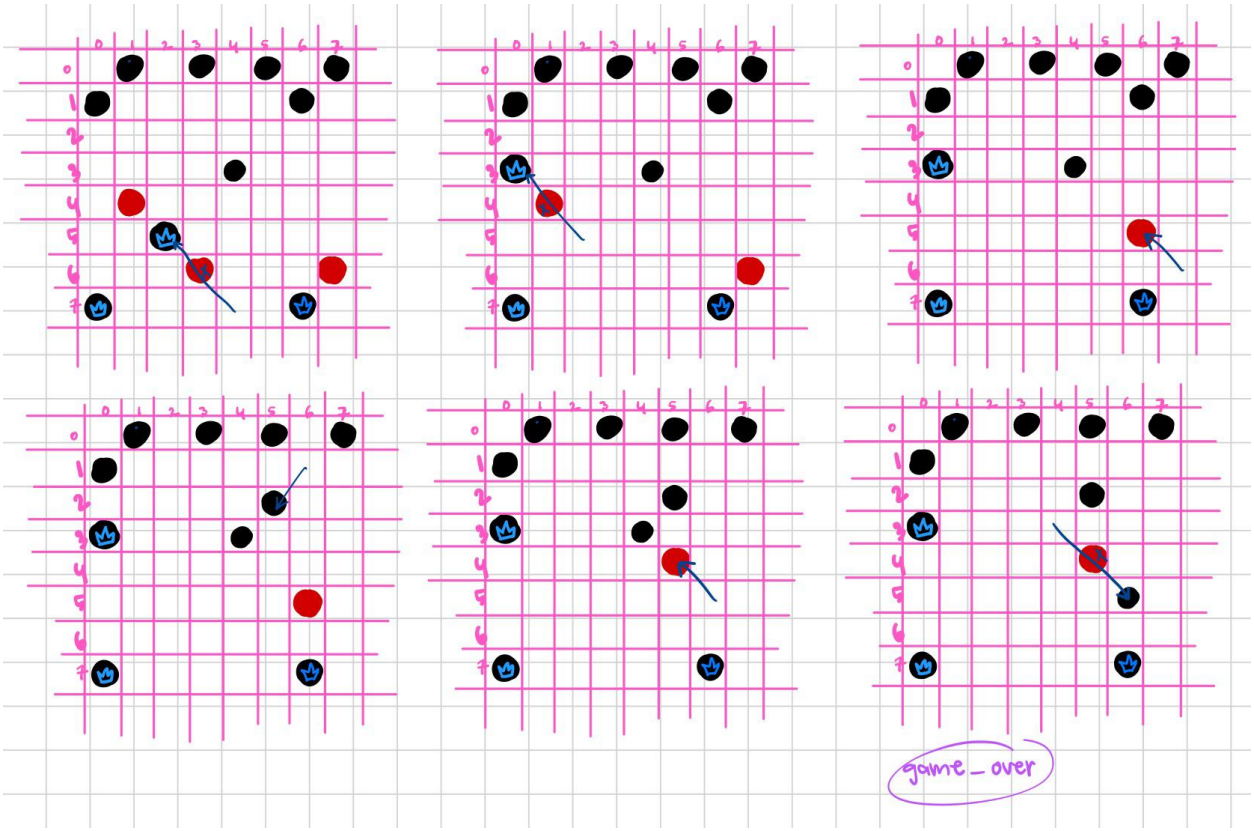


Figure 24

display_update.sv

The display_update module implemented our logic to create tiles on the VGA that corresponded to different checkerboard tiles. Again, the different states are as follows: empty dark brown tile, empty light brown tile, normal red token in dark brown tile, normal black token in dark brown tile, KING red token in dark brown tile, and KING black token in dark brown tile. We made red king tokens are purple and the black king tokens green. The testbenching for this display module is given in the figures below:

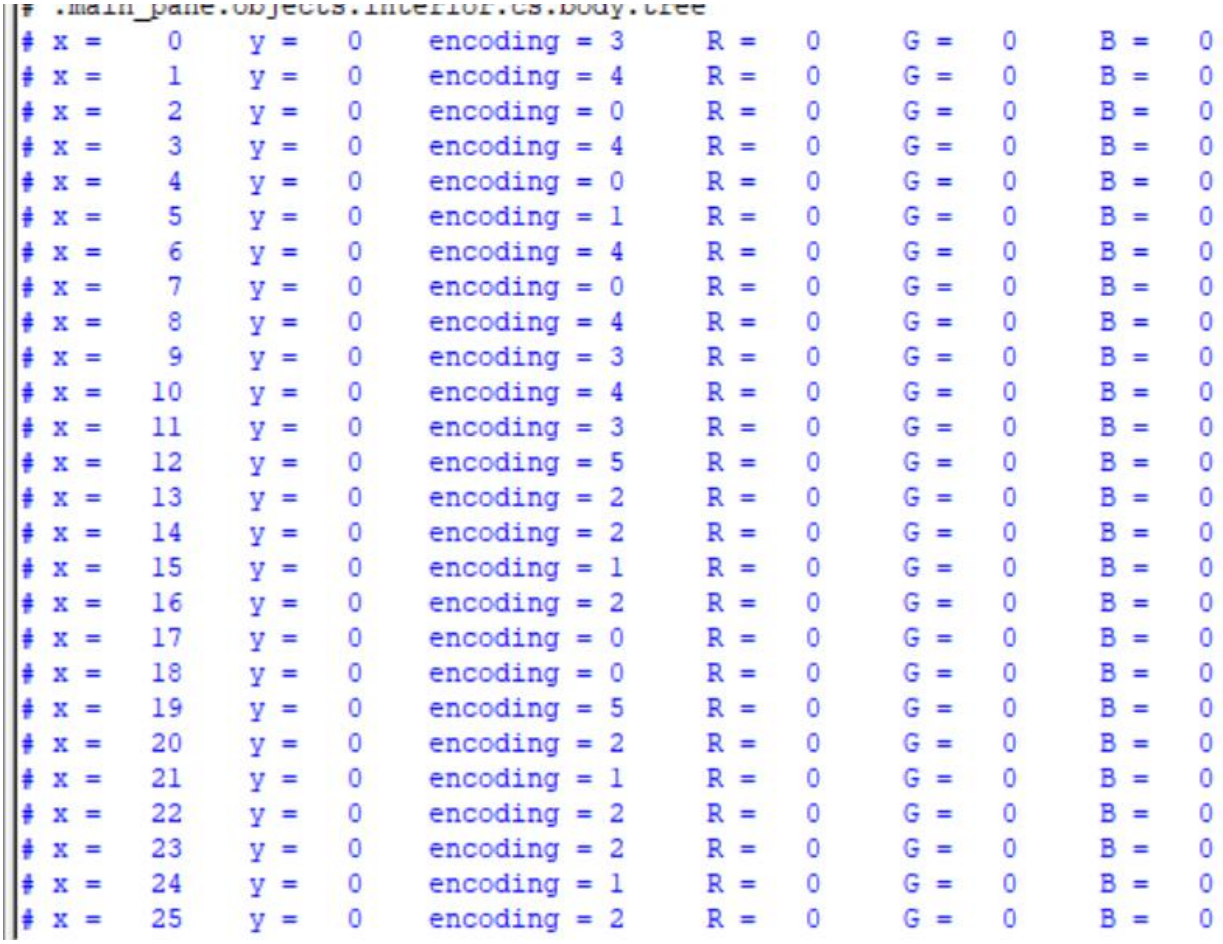
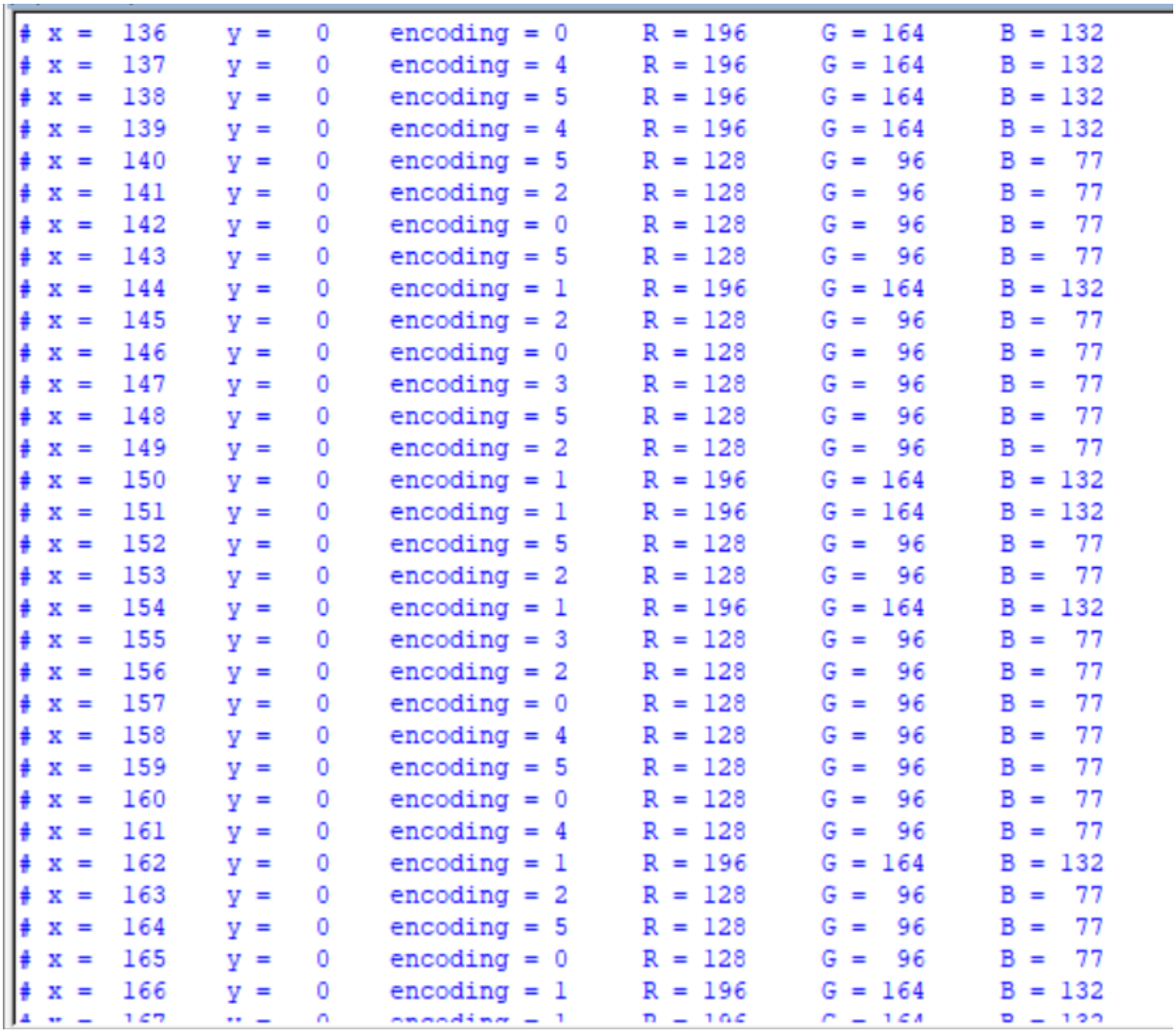


Figure 25: Black VGA screen (RGB = 000).

```
# x =   78    y =   0    encoding = 3    R =   0    G =   0    B =   0
# x =   79    y =   0    encoding = 2    R =   0    G =   0    B =   0
# x =   80    y =   0    encoding = 4    R = 196    G = 164    B = 132
# x =   81    y =   0    encoding = 3    R = 196    G = 164    B = 132
# x =   82    y =   0    encoding = 3    R = 196    G = 164    B = 132
# x =   83    y =   0    encoding = 1    R = 196    G = 164    B = 132
# x =   84    y =   0    encoding = 0    R = 196    G = 164    B = 132
# x =   85    y =   0    encoding = 3    R = 196    G = 164    B = 132
# x =   86    y =   0    encoding = 1    R = 196    G = 164    B = 132
# x =   87    y =   0    encoding = 5    R = 196    G = 164    B = 132
# x =   88    y =   0    encoding = 4    R = 196    G = 164    B = 132
# x =   89    y =   0    encoding = 5    R = 196    G = 164    B = 132
# x =   90    y =   0    encoding = 1    R = 196    G = 164    B = 132
# x =   91    y =   0    encoding = 0    R = 196    G = 164    B = 132
# x =   92    y =   0    encoding = 5    R = 196    G = 164    B = 132
# x =   93    y =   0    encoding = 5    R = 196    G = 164    B = 132
# x =   94    y =   0    encoding = 4    R = 196    G = 164    B = 132
# x =   95    y =   0    encoding = 3    R = 196    G = 164    B = 132
# x =   96    y =   0    encoding = 0    R = 196    G = 164    B = 132
# x =   97    y =   0    encoding = 5    R = 196    G = 164    B = 132
# x =   98    y =   0    encoding = 0    R = 196    G = 164    B = 132
# x =   99    y =   0    encoding = 5    R = 196    G = 164    B = 132
# x =  100    y =   0    encoding = 3    R = 196    G = 164    B = 132
# x =  101    y =   0    encoding = 4    R = 196    G = 164    B = 132
```

Figure 26: Different RGB values depending on the (x, y) values of the VGA display.



```
# x =  136    y =   0    encoding = 0    R = 196    G = 164    B = 132
# x =  137    y =   0    encoding = 4    R = 196    G = 164    B = 132
# x =  138    y =   0    encoding = 5    R = 196    G = 164    B = 132
# x =  139    y =   0    encoding = 4    R = 196    G = 164    B = 132
# x =  140    y =   0    encoding = 5    R = 128    G =  96    B =  77
# x =  141    y =   0    encoding = 2    R = 128    G =  96    B =  77
# x =  142    y =   0    encoding = 0    R = 128    G =  96    B =  77
# x =  143    y =   0    encoding = 5    R = 128    G =  96    B =  77
# x =  144    y =   0    encoding = 1    R = 196    G = 164    B = 132
# x =  145    y =   0    encoding = 2    R = 128    G =  96    B =  77
# x =  146    y =   0    encoding = 0    R = 128    G =  96    B =  77
# x =  147    y =   0    encoding = 3    R = 128    G =  96    B =  77
# x =  148    y =   0    encoding = 5    R = 128    G =  96    B =  77
# x =  149    y =   0    encoding = 2    R = 128    G =  96    B =  77
# x =  150    y =   0    encoding = 1    R = 196    G = 164    B = 132
# x =  151    y =   0    encoding = 1    R = 196    G = 164    B = 132
# x =  152    y =   0    encoding = 5    R = 128    G =  96    B =  77
# x =  153    y =   0    encoding = 2    R = 128    G =  96    B =  77
# x =  154    y =   0    encoding = 1    R = 196    G = 164    B = 132
# x =  155    y =   0    encoding = 3    R = 128    G =  96    B =  77
# x =  156    y =   0    encoding = 2    R = 128    G =  96    B =  77
# x =  157    y =   0    encoding = 0    R = 128    G =  96    B =  77
# x =  158    y =   0    encoding = 4    R = 128    G =  96    B =  77
# x =  159    y =   0    encoding = 5    R = 128    G =  96    B =  77
# x =  160    y =   0    encoding = 0    R = 128    G =  96    B =  77
# x =  161    y =   0    encoding = 4    R = 128    G =  96    B =  77
# x =  162    y =   0    encoding = 1    R = 196    G = 164    B = 132
# x =  163    y =   0    encoding = 2    R = 128    G =  96    B =  77
# x =  164    y =   0    encoding = 5    R = 128    G =  96    B =  77
# x =  165    y =   0    encoding = 0    R = 128    G =  96    B =  77
# x =  166    y =   0    encoding = 1    R = 196    G = 164    B = 132
```

Figure 27: RGB values for tokens within dark tiles.

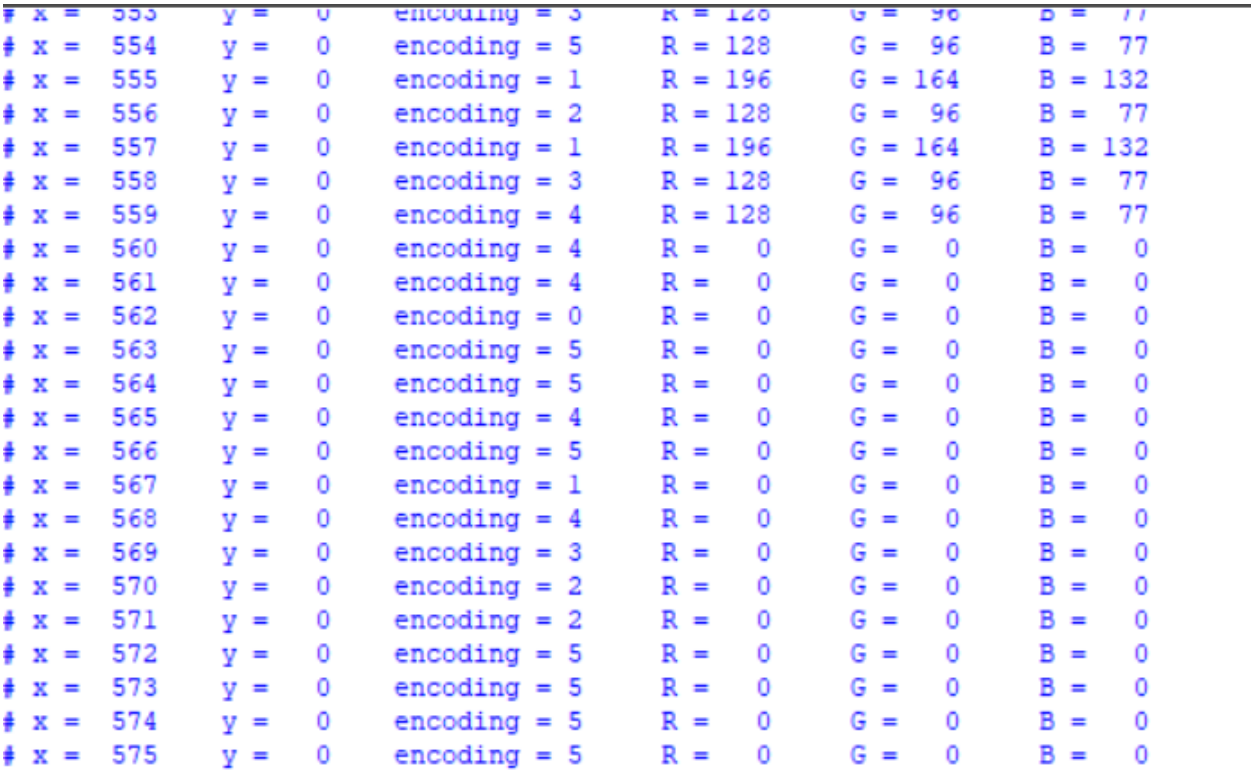


Figure 28: Black VGA screen for x > 559

The above Figures 25 through 27 show the RGB values that are fed into the VGA module (that we were provided with). The RGB values in Figure 25 and later half of Figure 28 are all 0s. This is because {R, G, B} = {0, 0, 0} corresponds to a black pixel, which is what we have set all the pixels less than x=80 and greater than x=559 to. In Figure 26, and 27, we see that when x is greater than 80 and less than 560, the RGB values are correspond to the encoding.

DE1_SoC.sv

While we did not extensively testbench the DE1_SoC top level module using waveforms (as opposed to testing with LabsLand), to ensure that the instances of our anti_metastability modules were working properly (two clock cycle delay from input to output). The anti-metastability modules are double FF synchronizers that we put the switches and keys through to deal with metastability. We also made sure that LEDR[0] was high at red's turn and LEDR[9] was high at black's turn, all the HEX values went low(since they're active low) at an invalid move, and at the end of the game, LEDRs [7:2] went high to signal the end. This is seen in Figures 28 through 31 below. We renamed LEDR[0] to "black's turn", LEDR[9] to "red's turn", and LEDR[7:2] to "game over LEDRs"

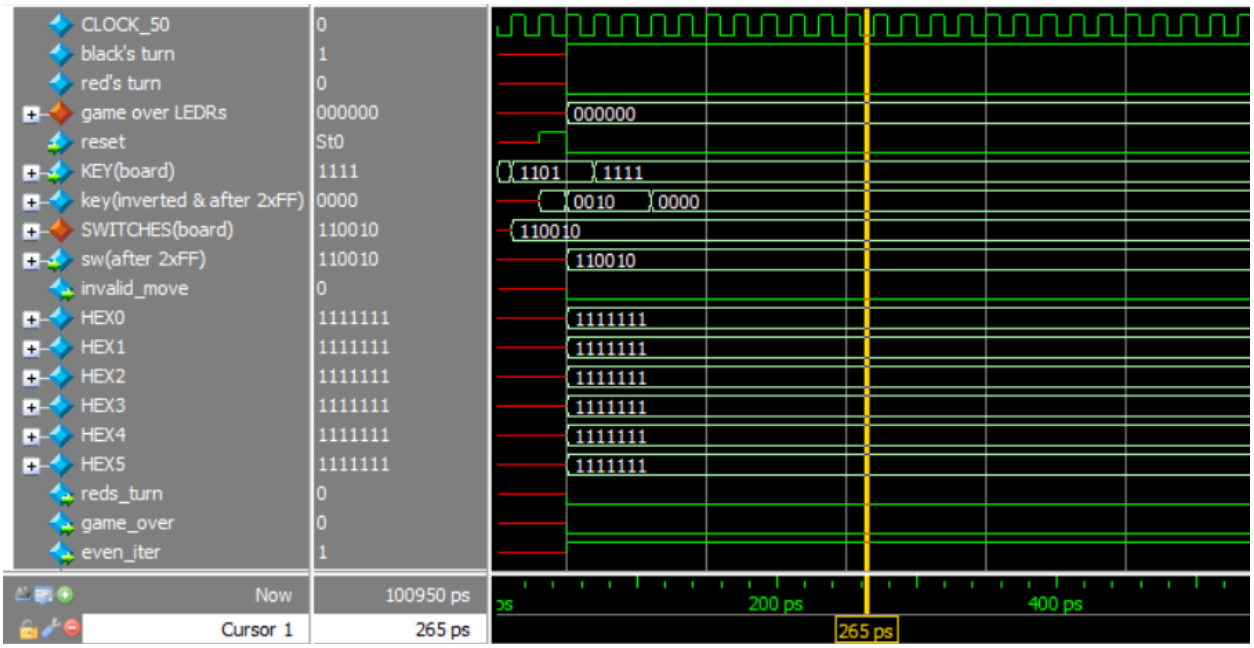


Figure 29: Double FF delay, black's turn = 0 and red's turn = 0 when reds_turn = 0

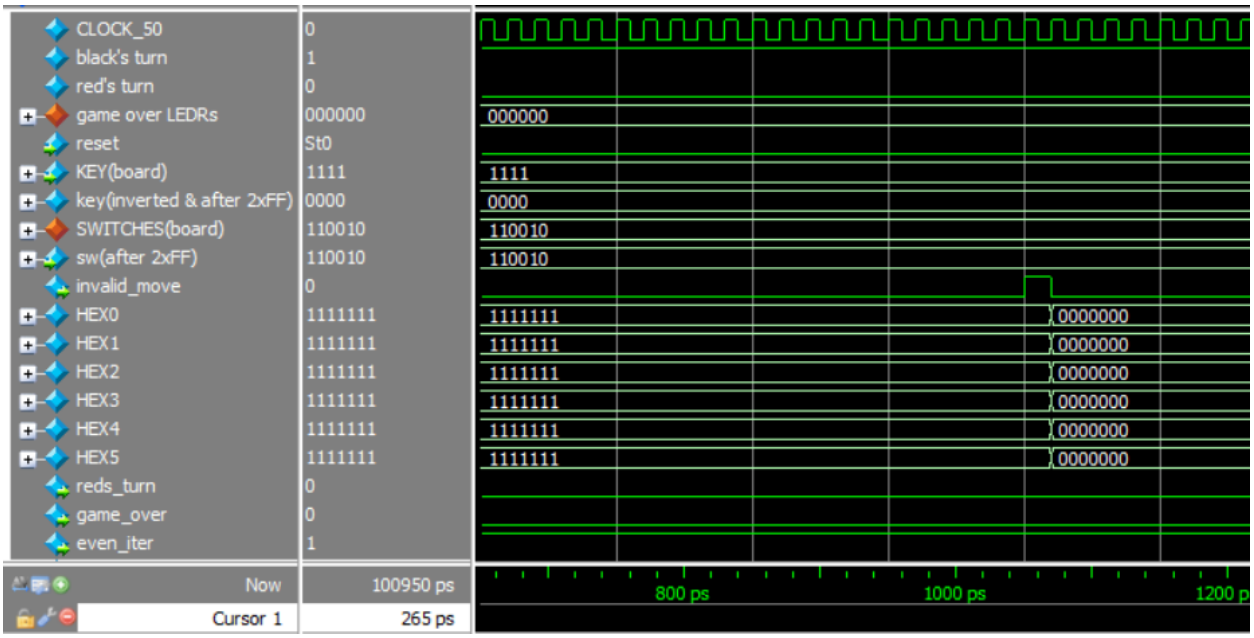


Figure 30: HEX displays flash at invalid_move

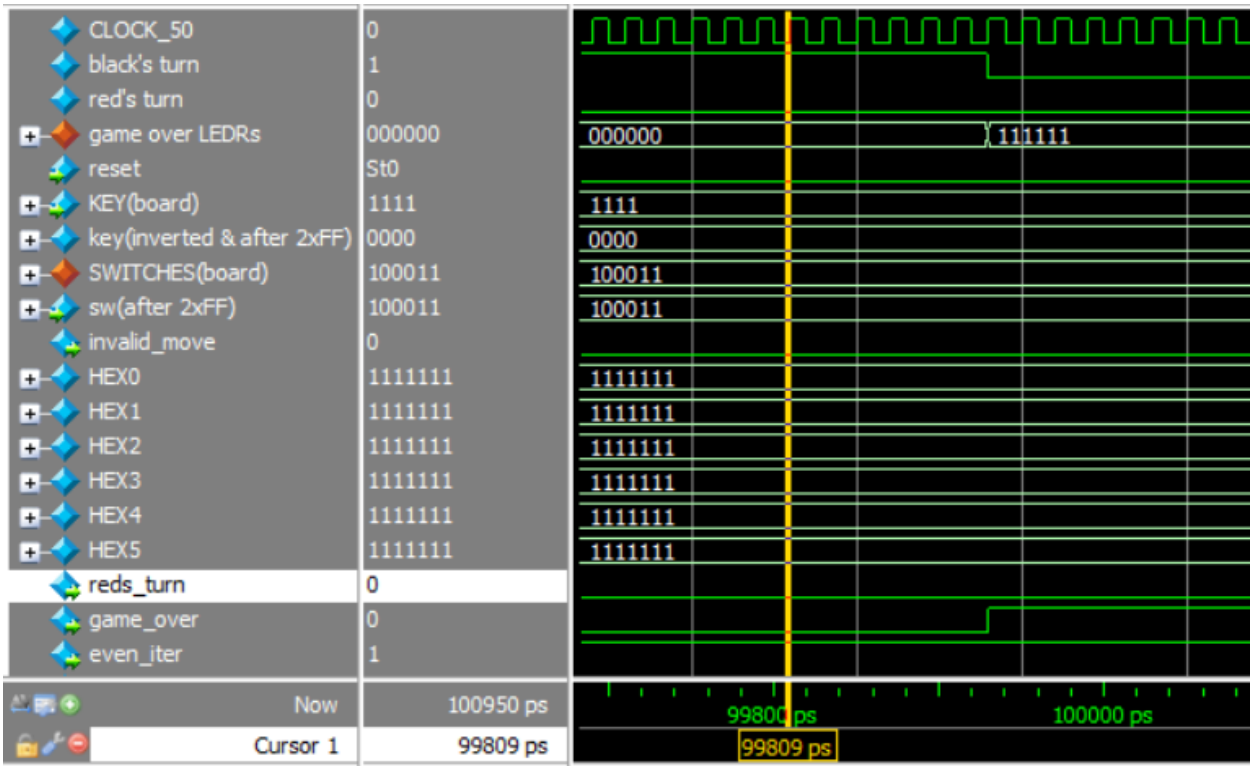


Figure 31: game over LEDRs flash at game_over

After looking at these waveforms, we moved on to testing the code using LabsLand directly.

The rest of the testing of the game is done using the FPGAs/VGAs available on LabsLand, where we tested all edge cases (including the ones we testbenched in ram_update_tb). By working through a game of checkers on the actual VGA display, we were able to tweak the tile and token colors, ensure that all switches and keys were working, and make sure that capture logic worked properly. An issue we ran into while testing on LabsLand at one point was that when making a move, even though the token was drawn at the new location, the token did not disappear from it's old location. This was an interesting issue to troubleshoot because where we update our 64x3 RAMs is in our second loop in our ASM, and during that time, we are using both write ports already to update the red encodings and black encodings for the new locations, and there is not a third port that we can use to erase the old location. Our solution was to, during the previous state at load_adj and load_skip, write in a dark square encoding into the 64x3 RAMs at the old location, which effectively erases the token from that spot. Another issue we had was that after skipping over a token, only one of the 64x3 RAMs updated with the erasure of the opponent's token, meaning at every iteration of the ASM, the board would alternate between showing a correctly erased tile at that location and an unerased token at that location. We needed to correctly update both 64x3 RAMs at once, but also needed to avoid a clash of a write and a

read at once by ram_update and display_update in the 64x3 RAMs. Our solution was to first store the location at which the skipped-over token needed to be erased from during load_skip, then later write in a blank dark brown tile encoding to the first port of both RAMs at the “read” control signal in the ASM. This works because even though we are writing to both RAMs at once, there wouldn’t be a read-write clash with the display_update module since only the second port is ever used for reads. Updating at the “read” control signals also ensures that writing in the encodings for the new locations(which takes up both write ports and is done at the check_done control signal) is done at separate times than this operation of erasing the skipped-over token.

Because LabsLand only allows us to use the FPGAs for a maximum of 3 minutes each, we were not able to test the game_over logic on the actual virtual FPGA/VGA display. Instead, we rely on the testbench of the ram_update to confirm that our game ends properly, especially since all other aspects of the game exactly match the ram_update testbench. This inability to test until the very end of the game is a drawback of our testing, but there was no other option when using LabsLand, as we were kicked out of each board before we could make more than 15-20 moves, and to end the game, one must capture all 12 tokens of the other team.

Other than this, we were able to test everything else, looking at waveforms, $display statements in the Transcript, and debugging by testing the functionality of the game on LabsLand. Overall, we have learned an incredible amount from this class, and are proud of the game we were able to implement for the EE371 final lab.

The Quartus Flow Summary is included below.

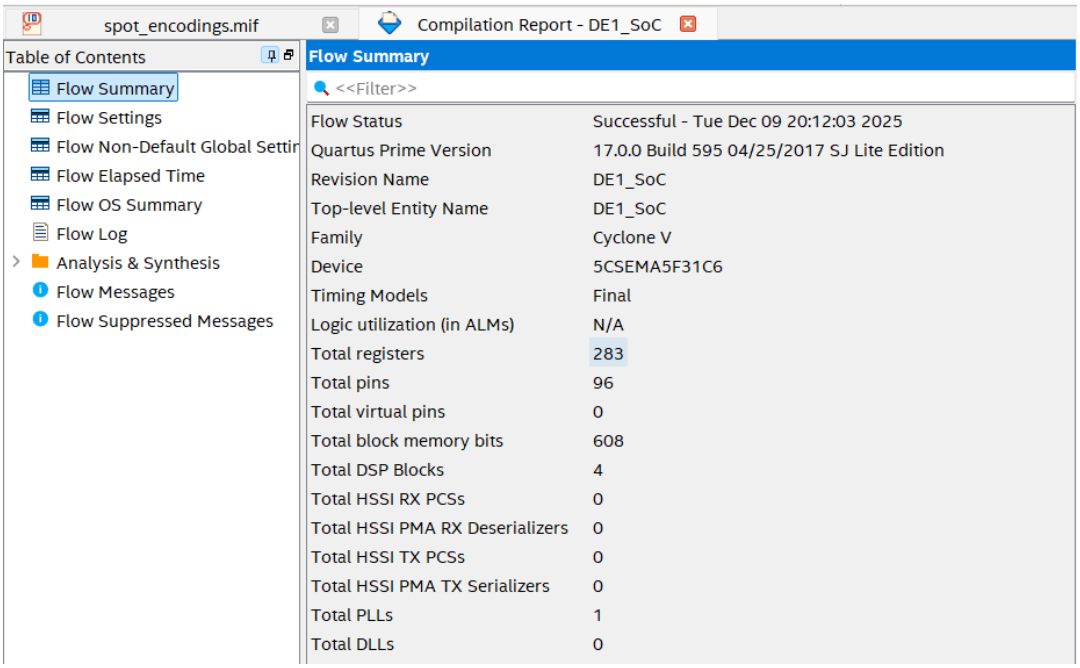


Figure 32: DE1_SoC flow summary

Experience Report

Project proposal: 3 hours

Initial planning: 1 hour

Creating ASMD: 6 hours

Implementing ASMD in code: 10 hours

Creating testbench for ASMD module: 6 hours

Testbenching and debugging from waveforms: 12 hours

Connecting RAM modules to VGA display: 8 hours

Testbenching video display: 3 hours

Debugging functionality of program using LabsLand: 10 hours

Lab report writing: 8 hours (interspersed within the rest of the coding/debugging)

Total: 67 hours

Work distribution:

Aya was in charge of writing ram_update and the connection of ram_update to the 64x3 RAMs. Brinda was in charge of the writing the display_update module as well as doing most of the testing for ram_update. We did a lot of brainstorming and debugging together and wrote the report together. Overall, the work was pretty much equally split.